



南京大學

本科畢業設計

學院 智能軟件與工程學院

專業 軟件工程（智能化軟件）

題目 基於 k-kick 哈希結構的冷熱分
層存儲設計與數據庫集成實驗

年級 2022 學 號 221900323

學生姓名 紀澤操

指導教師 劉明謀 職 稱 副教授

提交日期 2026 年 6 月 3 日



南京大学本科毕业论文（设计） 诚信承诺书

本人郑重承诺：所呈交的毕业论文（设计）（题目：基于 k-kick 哈希结构的冷热分层存储设计与数据库集成实验）是在指导教师的指导下严格按照学校和学院有关规定由本人独立完成的。本毕业论文（设计）中引用他人观点及参考资源的内容均已标注引用，如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文（设计）等违纪行为。

作者签名：纪泽操

学号：221900323

日期：2026 年 6 月 3 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：基于 k-kick 哈希结构的冷热分层存储设计与数据库集成实验

学院：智能软件与工程学院

专业：软件工程（智能化软件）

本科生姓名：纪泽操

指导教师（姓名、职称）：刘明谋 副教授

摘要：

k-kick 哈希相关理论表明，在插入和删除允许有限次元素搬移时，紧凑哈希表可以降低每键额外空间开销，并保持查询的常数时间复杂度。本文先实现该结构的简化原型，再在其上加入由热 tail cubby 和 SQLite 组成的冷热分层存储方案。基础原型保留 facility 分区、tail cubby、Local Query Router 和商压缩组件。该实现用于验证有限踢出、局部路由和键恢复能否在同一系统中配合运行。多层级 cubby 需要维护合并、拆分和删除回填状态，直接持久化每一层结构并不方便。因此，冷热分层方案将近期插入的数据暂存在内存 tail cubby；tail cubby 写满后，再以批量事务写入 SQLite，冷数据按键值记录管理。实验设置多层级内存结构、纯 SQLite 结构和冷热分层结构三类对照。各组使用相同键集合、访问序列和统计口径，比较操作延迟与平均每键空间占用。结果显示，在写后读和近期访问负载下，内存热层缩短了部分查询路径；删除近期键时，也减少了多层级回填。与纯 SQLite 相比，冷热分层方案的查询延迟、删除延迟和按本文口径统计的每键空间占用更低，但部分规模下写入延迟并不占优。

关键词：k-kick 树；tail cubby；冷热分层存储；数据库集成

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Design of Hot-Cold Tiered Storage Based on k-kick Hashing and Database Integration Experiments

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software Engineering (Intelligent Software)

UNDERGRADUATE: Ji Zecao

MENTOR: Liu Mingmou, Associate Professor

ABSTRACT:

The theory of k-kick hashing shows that, when insertion and deletion allow a bounded number of element moves, a compact hash table can reduce auxiliary space per key while keeping constant-time queries. This thesis first implements a simplified prototype of that structure, then adds a hot-cold storage scheme built from an in-memory tail cubby and SQLite. The base prototype keeps facility partitioning, the tail cubby, the Local Query Router, and quotient compression. These components are used to test whether bounded kicking, local routing, and key recovery can work together in one system. Persisting the original multi-tier cubby structure directly is awkward, because merging, splitting, and deletion backfilling all depend on current tier state. The hot-cold design therefore keeps recently inserted keys in the in-memory tail cubby; when the tail becomes full, it flushes valid keys to SQLite in a batch transaction, and cold keys are managed as database records. The experiments compare the multi-tier in-memory structure, a pure SQLite structure, and the hot-cold tiered structure under the same key sets, access sequences, and metric definitions. Under write-after-read and recent-access workloads, the memory hot layer shortens part of the query path, and deleting recent keys triggers less multi-tier backfilling. Compared with pure SQLite, the hot-cold scheme has lower query latency, deletion latency, and per-key space usage under the measurement method used here, but its insertion latency is not lower at all scales.

KEYWORDS: k-kick tree; tail cubby; hot-cold tiered storage; database integration

目 录

目 录	V
第一章 引言	1
1.1 实验背景及意义	1
1.2 国内外研究现状	2
1.3 本文主要工作	3
1.4 本文组织结构	3
第二章 问题模型与理论基础	5
2.1 探测复杂度与球槽分配模型	5
2.2 系统核心设定：全域与哈希基础	6
2.2.1 全局参数	6
2.2.2 可逆置换与全局路由哈希	6
2.2.3 facility 编号与 cubby 内局部地址	7
2.3 核心组件层级关系	7
2.4 商压缩的信息论思想	8
2.5 k-kick、MiniArray 与 Local Query Router	9
2.5.1 k-kick：逻辑分层与交换上界	9
2.5.2 MiniArray M	10
2.5.3 Local Query Router: $O(1)$ 查询与映射存储	10
2.5.4 辅助结构同步不变量	10
2.6 本章小结	11
第三章 哈希表结构与算法设计	13
3.1 facility 与 MiniArray	13

3.1.1	facility 的基本属性	13
3.1.2	MiniArray 的逻辑结构	13
3.1.3	Access: 按逻辑下标读取	14
3.1.4	Insert: 插入与叶分裂	14
3.1.5	Delete: 删除与叶合并	14
3.1.6	Update: 原地覆写	16
3.1.7	Split 与 Merge	16
3.1.8	Rank 与 Select	16
3.1.9	复杂度与空间性质	17
3.2	多层次 cubby 设计	17
3.2.1	层级属性与初始状态	17
3.2.2	动态调整: 合并与拆分	18
3.2.3	cubby 内部组件	18
3.2.4	插入与删除操作	19
3.3	k-kick 层级插入算法	19
3.3.1	层级子区间定义	20
3.3.2	偏好序列	20
3.3.3	探测位置序列	20
3.3.4	子区间饱和与最大可用深度	21
3.3.5	随机深度选择与 InsertKick	22
3.3.6	ReInsert: 被踢元素向上放入	22
3.3.7	复杂度与正确性	24
3.4	Local Query Router	24
3.4.1	设计目标	24
3.4.2	映射规则	25
3.4.3	二叉前缀树存储与接口	25
3.4.4	复杂度与 Patricia 压缩	27
3.5	商压缩与键恢复	27
3.5.1	设计动机与信息分工	27
3.5.2	$\pi(x)$ 与 $g^*(x)$ 的位段划分	28

3.5.3	MiniArray B 中的 MetaEntry	28
3.5.4	恢复: 重建 $\pi(x)$	29
3.5.5	查询校验	29
3.6	插入、查询与删除	30
3.7	冷热分层键存储实现	30
3.8	本章小结	32
第四章	系统性能实验	37
4.1	实验目的与环境	37
4.2	实验方案	37
4.3	规模 n 实验	38
4.4	facility 规模 K 实验	39
4.5	k-kick 深度 k 实验	40
4.6	多层级 cubby 动态重构实验	40
4.7	本章小结	42
第五章	冷热分层存储对照实验	43
5.1	实验目的与方案设置	43
5.2	负载与统计口径	43
5.3	多层级内存结构对照	44
5.4	纯数据库结构对照	44
5.5	空间占用与正确性对照	45
5.6	本章小结	46
第六章	总结与展望	47
6.1	工作总结	47
6.2	不足与局限	47
6.3	展望	48
参考文献		49
致 谢		55

第一章 引言

1.1 实验背景及意义

哈希表通过哈希函数把关键字映射到数组位置^[1]，常用于数据库索引、键值存储、编译器符号表和运行时系统^[2-3]。在这类场景中，查询路径通常较短，工程实现也相对成熟；但当数据规模扩大后，哈希表为维持查询和更新所保存的辅助信息会逐渐占据不可忽略的空间。系统除存放键或键的摘要外，还要记录冲突处理、删除标记、元素重定位和键恢复所需的元数据。面向键值存储的哈希索引优化也是国内已有工作的关注方向之一，相关研究从键摘要压缩、热点感知调度等角度改进空间利用和探查路径^[4]。因此，评价哈希表时需要同时考察平均查询时间和每个键背后的元数据成本。

相较于链地址法，开放寻址哈希表把元素直接放在连续数组中，查询时不需要沿指针跳转，也更容易利用缓存局部性。它的代价也比较明确：负载因子升高后，冲突和探测长度会增加，工程实现往往需要预留空槽来降低冲突概率。删除操作还会带来额外限制。删除后的空槽如果直接释放，可能破坏后续查询路径；若采用回填或重定位，又必须维护被移动元素的状态。本文关注的核心问题由此形成：在查询、插入和删除时间可控的前提下，尽量降低元素定位、更新和键恢复所需的辅助存储，并观察相关理论结构落到原型系统后产生的维护成本。

Bender 等人的论文^[5] 从理论层面给出了更细的刻画：若插入和删除允许承担 $O(k)$ 次元素交换代价（一次更新过程中被搬移的元素个数），则每个元素的额外空间可从 $O(\log \log n)$ 比特降至 $O(\log^{(k)} n)$ 比特，同时查询时间仍保持 $O(1)$ 。本文空间开销均按比特计， $\log^{(k)} n$ 表示对 n 连续取 k 次对数。这个结论说明，开放寻址哈希表的空间成本同时受到空位占比、元素搬移、局部路由信息和原始键恢复编码的影响。本文围绕该结构完成原型实现和实验分析，用以观察理论权衡进入工程实现后会出现哪些具体开销。

1.2 国内外研究现状

常见哈希表实现可粗略分为链地址法与开放寻址法^[1]。链地址法借助链表或桶结构处理冲突，实现直接，插入和删除也容易理解；不足在于每个元素可能需要额外指针，内存分配也会破坏连续访问。开放寻址法将元素放入数组槽位，局部性较好，却更容易受负载因子、探针序列和删除策略影响。线性探测及其改进形式通过连续探查换取更好的缓存行为^[6-7]，Robin Hood 哈希等方法则关注不同元素探测距离的均衡^[8]。这类方法的共同难点在删除和高负载场景：墓碑标记会延长查询路径，立即回填则要求被移动元素的查询路径仍然可恢复。

布谷鸟哈希^[9-10]通过多候选位置与踢出过程缩短查询路径，但在高负载下仍需要处理踢出链和重哈希；动态完美哈希^[11]则给出了最坏情况常数查询时间的经典上界。最小完美哈希^[12-16]面向静态集合提供更强查找保证，适合关键词集合固定、查询远多于更新的场景。围绕最小完美哈希的后续研究继续优化建表代价、编码空间和种子搜索过程^[17]。这些结构在静态数据上空间效率较高，但若应用到频繁插入删除的键值存储，还需要额外机制处理增量更新和失败重构。

仅靠负载因子无法刻画紧凑哈希表的真实成本，因为系统还须保存支持查询、更新与键恢复的辅助结构。近似成员查询常用的 Bloom 过滤器^[18-19]与动态检索、过滤器的下界^[20]说明，支持更新的紧凑结构需要为状态变化付出额外信息量；紧凑检索结构^[21]则展示了如何在较小空间内保存与键相关的值。Bender 等人的最优时间/空间权衡框架^[5]将元素放置建模为带删除的球槽分配^[22-23]，并以 k-kick、Local Query Router、MiniArray 与商压缩组合出可调的权衡曲线。本文的基础原型主要围绕这一方向展开。

工程系统中的关注点略有不同。打包与压缩哈希表^[24]、IcebergHT^[25-26]等方案直接面向真实机器评价实现效果。除吞吐和装填率外，它们还要处理缓存行为、重构代价等实现因素。国内方面，面向持久化键值数据库的哈希索引优化^[4]、GPU 环境下的动态哈希表索引^[27]等工作分别从热点感知、键摘要压缩和分页显存管理等角度降低索引开销。与这些工程方案相比，本文不追求完整替代成熟哈希表实现，而是把理论结构中的关键模块连成可运行原型，再通过实验观察有限踢出、局部路由和层级维护在实际执行中的开销。

1.3 本文主要工作

本文在 Bender 等人的框架下实现 facility-cubby-Slot 分层结构的简化原型，完整理论结构中的部分细节则按原型实验需要作了工程简化。facility 负责表级分区，cubby 负责局部存储和层级调度，Slot 存放商压缩后的载荷。MiniArray 用于维护变长元数据；cubby 内的 k-kick 完成有界交换插入；Local Query Router 将查询从探测序列扫描转化为局部映射访问；MiniArray B 保存键恢复所需的差分信息。该原型保留地址拆分、有限踢出、局部路由和键恢复四个环节。借助同一实现，可以分别测量查询路径、元素搬移和层级重构带来的开销。

原始多层级 cubby 结构在落地时需要维护层级合并、拆分和删除回填。在完成基础原型后，本文进一步补充冷热分层键存储方案：tail cubby 作为近期数据入口，写满后把较冷的键批量下沉到 SQLite。这个改造借用了日志结构合并树先写内存、再批量下沉的思路^[3,28]，但不再完整保留原结构中多层级 cubby 的冷数据管理方式。第四章用于检验多层级原型结构是否能按规则运行，第五章则单独讨论冷热分层改造与两个对照结构之间的差异。

围绕上述实现，实验部分关注四类问题：(1) 理论模块如何落到具体数据结构，并在插入、查询、删除过程中保持状态一致；(2) MiniArray、k-kick 与 Local Query Router 联合实现后，插入、查询与删除各阶段的延迟处于何种量级；(3) 在不同的 n 、 K 、 k 分组实验下，k-kick 踢出次数、cubby 拆分与合并次数等指标如何变化，单次搬移次数是否仍受 k 控制；(4) 冷热分层改造后，内存 tail cubby 与 SQLite 冷数据层在写入、查询、删除和空间占用之间形成怎样的取舍。

1.4 本文组织结构

后续章节先说明理论基础和结构实现，再进入实验。第二章给出球槽分配模型、全局路由、商压缩和同步不变量；第三章说明 facility、MiniArray、cubby、k-kick、Local Query Router 与键恢复过程的实现细节。第四章测量多层级原型结构在不同参数下的延迟、踢出和层级重构行为；第五章讨论冷热分层改造在写入、近期查询、近期删除和空间占用上的表现。第六章总结本文工作，并说明原型实现仍存在的限制。

第二章 问题模型与理论基础

2.1 探测复杂度与球槽分配模型

开放寻址哈希表可被抽象为在线状态下的球槽分配问题^[22-23,29]：表中有固定数量的槽位，每个槽位至多容纳一个元素；系统需要支持动态插入与删除，且任一时刻元素个数不超过槽位数。对任意键 x ，哈希过程会为其生成一条预先确定的随机探测序列

$$h_1(x), h_2(x), \dots \quad (2-1)$$

元素只会被安置于该序列给出的候选槽位之一。

当某元素最终被置于其探测序列的第 i 个候选位置时，探测复杂度（查询时为记录“元素落在第 i 个候选位置”所需的比特数，记为 $\Theta(1 + \log i)$ ，与辅助元数据空间直接相关）定义为 $\Theta(1 + \log i)$ 。该量可理解为查询阶段为记录“元素采用了第几个候选位置”而必须保存的比特数目，因而与辅助元数据的空间开销直接相关。在该模型下，用平均探测复杂度刻画整体附加空间，用交换成本刻画单次更新中被搬移元素的个数；最优哈希表理论所关注的核心问题，是在允许有限交换的前提下，如何尽可能降低平均探测复杂度^[5-6,20]。一般而言，若表内 n 个元素分别落在其探测序列的第 $i_1, i_2, \dots, i_n$ 个候选位置，则平均探测复杂度为

$$\frac{1}{n} \sum_{i=1}^n \Theta(1 + \log i_i), \quad (2-2)$$

而单次插入或删除的交换成本为踢出链中被搬移元素个数的上界。Bender 等人^[5]通过调节 k-kick 深度参数 k ，使上述两项复杂度分别控制在 $O(\log^{(k)} n)$ 与 $O(k)$ 量级，形成可调的时间/空间权衡曲线。他们提出：若查询必须在常数时间内完成，且插入、删除允许承担 $O(k)$ 次元素交换，则每个元素所需的额外空间可由 $O(\log \log n)$ 比特降至 $O(\log^{(k)} n)$ 比特。该结果说明，开放寻址哈希表的空间效率受空位占比、元素搬移、局部引导信息及键恢复编码共同影响。后文将给出与之

对应的地址拆分、分层存储与 Local Query Router 设计，使探测序列在插入阶段完整定义、同时满足查询阶段不必线性扫描。

2.2 系统核心设定：全域与哈希基础

2.2.1 全局参数

设键空间全域为 U ，且 $|U| = \text{poly}(n)$ 。当前表内元素规模记为 n ；表容量 N 取为 2 的幂，使实际容量落在区间 $[N, 2N]$ 内。每个 facility 的基准规模为

$$K = \text{polylog } n, \quad (2-3)$$

通常取 $K = \Theta(\log^c n)$ 且 $c \in [2, 4]$ ，以便在 facility 数量 N/K 与局部结构规模之间取得平衡^[1,5]。

2.2.2 可逆置换与全局路由哈希

对任意键 x ，先经随机可逆置换函数 π 映射为 $\pi(x)$ 。该类置换可由 Feistel 等构造从伪随机函数生成^[7,30-31]。定义 $x[a, b]$ 表示 x 的第 a 位到第 b 位组成的子串（从低位到高位），则全局路由哈希 $g^*(x)$ 定义为 $\pi(x)$ 的低 $\log N$ 位：

$$g^*(x) = \pi(x)[1, \log N]. \quad (2-4)$$

置换值在 $\log N$ 位以上的部分存入槽位载荷，记为

$$\text{storage} = \pi(x)[\log N + 1, \log U], \quad (2-5)$$

即商压缩意义下的高位载荷；该部分不再参与 facility 与 cubby 的路由，仅在命中校验时与局部元数据联立以恢复完整的 $\pi(x)$ 。

从比特布局上看， $\pi(x)$ 可示意划分为“高位载荷 + 低位 $g^*(x)$ ”；而 $g^*(x)$ 自身又可自高向低继续划分为 facility 编号段、cubby 中间段与 Local Query Router 段，分别用于表级分区、cubby 内桶化与 $A[i]$ 索引。图 2-1 概括了这一层次。

$\pi(x)$ 高位: 载荷	$g^*(x) = \pi(x)[1, \log N]$	
$\pi(x)[\log N + 1, \log U]$	facility 位	cubby 中间位 + $g_I(x)$
存入槽位数组	$r = g^*[\log K + 1, \log N]$	$g^*[1, \log K]$

图 2-1 $\pi(x)$ 与 $g^*(x)$ 的位段划分

2.2.3 facility 编号与 cubby 内局部地址

在 $g^*(x)$ 已确定的前提下, facility 编号取

$$r = g^*(x)[\log K + 1, \log N] = \left\lfloor \frac{g^*(x)}{K} \right\rfloor. \quad (2-6)$$

对容量为 $|I|$ 的 cubby, 其内部 Local Query Router 段 (与 k-kick 偏好序列无关) 定义为

$$g_I(x) = g^*(x)[1, \log |I|], \quad (2-7)$$

即 $g^*(x)$ 的最低 $\log |I|$ 位, 取值范围为 $[0, |I| - 1]$ 。所有满足 $g_I(x) = i$ 的键由该 cubby 内第 i 号 Local Query Router $A[i]$ 管理; $g^*(x)$ 中 $[\log |I| + 1, \log K]$ 的中间位则与 facility 编号 r 及高位载荷共同用于恢复完整 $g^*(x)$, 进而经由 π^{-1} 得到原始键 $x^{[5,21]}$ 。

2.3 核心组件层级关系

在逻辑结构上, 整张表可视为一个大规模数组, 均等划分为 (N/K) 个 facility; 每个 facility 管理规模为 K 的局部范围, 并在其内部维护多层级、不同容量的 cubby 集合。cubby 是实际存放元素的局部块, 其内部状态可以分为五类: (1) 槽位数组, 长度为 $|I|$, 存放商压缩后的高位载荷 $\pi(x)[\log N + 1, \log U]$; (2) k-kick 逻辑, 将 cubby 按深度划分为若干逻辑子区间, 用于插入、踢出和被踢元素重插; (3) MiniArray A , 长度为 $|I|$, 作为 Local Query Router 数组维护 $g_I(x) = i$ 的键到探测序列下标的映射; (4) MiniArray M , 记录各深度子区间的空闲槽位, 使插入与重插不必扫描整个 cubby; (5) MiniArray B , 与槽位数组一一对应, 保存 $\delta = g_I(x) - i$ 以及恢复 $g^*(x)$ 所需的中间位。

facility 层另维护一棵基于 B 树、深度为 $O(1)$ 的 MiniArray, 用于在变长元数据与 facility 级索引上实现均摊常数时间的访问、插入、删除与更新^[5,32]。

MiniArray 中每个内部节点最多有多少个子节点称为扇出，取 $\Theta(\log n / \log \log n)$ ，内部指针仅需 $\Theta(\log \log n)$ 比特，从而到达整体 $O(\log^{(k)} n)$ 的空间目标^[33-34]。

为在动态扩缩容下维持高装填因子，每个 facility 还引入 tail cubby 机制：初始时仅含一个第 1 层 cubby 作为 tail cubby；此后所有新插入默认进入 tail cubby，tail cubby 满后转为普通第 1 层 cubby 并新建 tail cubby。不同 j 层 cubby 的目标个数与容量分别为

$$t_j = \Theta\left(\frac{(\log^{(j+1)} n)^2}{(\log^{(j)} n)^2}\right), \quad r_j = \frac{K}{(\log^{(j)} n)^2}; \quad (2-8)$$

当某层 cubby 数量偏离 t_j 时，通过向上合并或向下拆分并在新 cubby 内重插全部元素来恢复分布不变量。删除非 tail cubby 元素时，从 tail cubby 随机抽取一个元素回填至被删槽位；tail cubby 为空时从第 1 层提升一个 cubby 作为新 tail cubby，并可能触发拆分。层级调度、tail cubby 不变量与 k-kick 踢出规则共同构成第三章的算法主体^[1,5,8,25]。

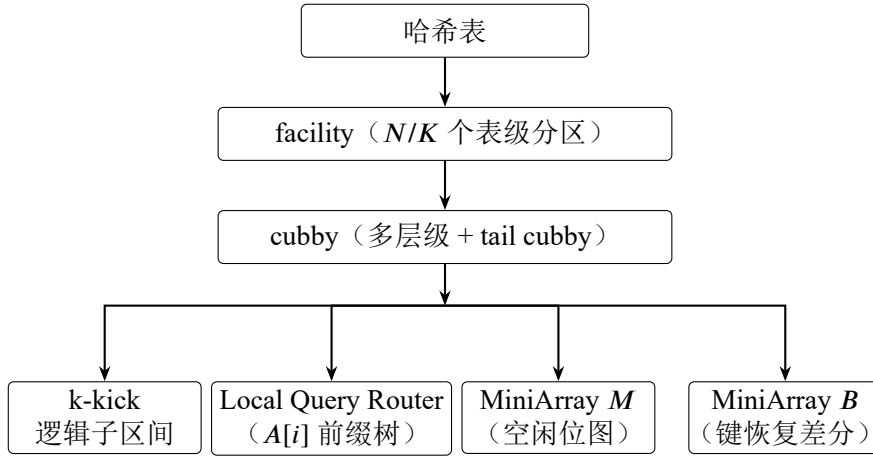


图 2-2 facility-cubby-Slot 分层架构

2.4 商压缩的信息论思想

如果在槽位中完整保存原始键，则部分信息与寻址过程重复。商压缩借助双射函数 π ，映射 x 为 $\pi(x)$ ，低 $\log N$ 位 $\pi(x)[1, \log N] = g^*(x)$ 的各位段交由地址上下文（facility 编号、cubby 容量、槽位下标）与局部元数据分担，槽位仅保存高位载荷 $\pi(x)[\log N + 1, \log U]$ 与无法由上下文恢复的差分信息。

具体地，当槽位数组第 i 位存放键 x 对应信息时，MiniArray $B[i]$ 保存

$$\delta = g_I(x) - i, \quad (2-9)$$

以及 $g^*(x)[\log |I| + 1, \log K]$ 的比特位。配合 facility 编号 $r = g^*(x)[\log K + 1, \log N]$ ，即可重建

$$g^*(x)[1, \log N], \quad (2-10)$$

再加上高位载荷得到 $\pi(x)$ ，经 π^{-1} 恢复 x 。紧凑检索结构^[12,21]和打包式哈希表^[14,24]都关注“按条目实际长度计费”的空间口径。本文采用的结构还要处理 k-kick 动态搬移与层级重建，因此 B 、 A 与 M 必须随槽位变化同步更新；这也是 MiniArray 与 Local Query Router 的动态维护比静态紧凑哈希更复杂的原因^[5,20]。

2.5 k-kick、MiniArray 与 Local Query Router

2.5.1 k-kick：逻辑分层与交换上界

在 cubby 内部，k-kick 并不引入物理上的多层数组，而是将槽位数组上长度为 $|I|$ 的连续槽位按固定规则划分为 $k + 1$ 个逻辑深度：深度 0 的子区间覆盖整个 cubby，更深层子区间由上一层均匀切分得到，规模 $s_i = \Theta((\log^{(i)} K)^6)$ 。对键 x （此处均指 $\pi(x)$ ）定义偏好序列

$$g_0(x) \rightarrow g_1(x) \rightarrow \cdots \rightarrow g_k(x), \quad (2-11)$$

其中 $g_0(x)$ 为整个 cubby， $g_{i+1}(x)$ 为 $g_i(x)$ 经均匀切分后 x 所属的子区间。探测位置序列 $h_1(x), h_2(x), \dots$ 按先深后浅顺序枚举各子区间内槽位；若元素最终落在第 j 个候选位置，则其探测复杂度为 $\Theta(1 + \log j)$ 。

定义逻辑子区间在已满且其中每个元素的 `insert_depth` 均不低于该子区间对应深度时饱和（`IsSaturated`）；贪心策略下 `GetMaxUsableDepth(x)` 返回 x 可插入的、未饱和子区间的最大深度。插入时，系统先取随机哈希 $s(x) \in [0, k]$ ，再令

$$\text{depth} = \min(\text{GETMAXUSABLEDEPTH}(x), s(x)), \quad (2-12)$$

在子区间 $g_{\text{depth}}(x)$ 内写入；若该子区间已满，则踢出其中 insert_depth 最小的元素并调用 ReInsert 向上安置。Bender 等人^[5] 证明：在参数满足其设定时，整条踢出链至多 k 次，单次插入的交换成本为 $O(k)$ ；配合随机深度选择，平均探测复杂度可控制在 $O(\log^{(k)} n)$ 量级，从而与上述描述的空间–时间权衡曲线对齐^[6,9-10]。

2.5.2 MiniArray M

k -kick 的 IsSaturated 、 GetMaxUsableDepth 与 ReInsert 都需要在逻辑子区间上判断是否存在空位并找到可用位置。MiniArray M 为各深度子区间维护等价于位图的空闲槽位图^[35]，使上述判定与定位在均摊意义下为常数时间，而不必扫描整个 cubby ^[5-6]。MiniArray 则通过 $O(1)$ 深度 B 树、叶内 Rank/Select 与紧凑叶节点维护变长元数据，并实现常数时间复杂度的 $\text{Access/Insert/Delete/Update}$ 函数^[32,34]：树高 $O(1)$ ，内部指针 $\Theta(\log \log n)$ 比特^[13,24]。

2.5.3 Local Query Router: $O(1)$ 查询与映射存储

偏好序列与探测位置序列在插入阶段完整定义；为实现 $O(1)$ 查询，须额外维护键到探测下标的映射，而不遍历 $h_1(x), h_2(x), \dots$ ： k -kick 插入会搬移元素，其实际位置既不等于首选位置 $g_k(x)$ ，也不等于 $h_1(x)$ 。Local Query Router 因此为每个键保存映射

$$\pi(x) \longrightarrow j, \quad (2-13)$$

其中 j 为探测序列下标。 cubby 内令 $g_I(x) = g^*(x)[1, \log |I|]$ ，所有满足 $g_I(x) = i$ 的键由路由器 $A[i]$ 管理； $A[i]$ 实现为二叉前缀树（可进一步 Patricia 压缩^[17,36]），对外接口为 $\text{insert}(\text{key}, j)$ 、 $\text{query}(\text{key})$ 、 $\text{delete}(\text{key})$ 。查询时先算 $i = g_I(x)$ ，在 $A[i]$ 得 j ，再访问槽位 $h_j(x)$ ，加上 $B[h_j(x)]$ 与高位载荷恢复 $\pi(x)$ ，与查询侧的 $\pi(x)$ 进行对比； $j \leq \log K$ ，每条映射仅需 $O(\log \log n)$ 比特，查询路径为常数次路由器与 MiniArray 访问^[5,21]。

2.5.4 辅助结构同步不变量

元素在 k -kick 踢出链中每发生一次搬移，须同步更新四处状态：(1) 槽位内容与 insert_depth ；(2) $B[i]$ 中的 δ 与中间位；(3) $A[g_I(x)]$ 中 $\pi(x) \mapsto j$ 的映射；(4)

M 中各子区间的空闲位图。删除非 tail cubby 元素时的 tail cubby 随机回填、层级合并/拆分后的全量重插, 以及表级双表扩缩容, 均遵循同一同步规则——不可以仅复制高位载荷而忽略路由器与 B 的上下文依赖^[1,5]。第三章将给出 MiniArray、k-kick 与 Local Query Router 的全部算法伪代码及统一操作语义。

2.6 本章小结

本章建立了后文所需的理论语境: 球槽分配模型给出探测复杂度与交换成本的定义^[22]; 全域参数与 $g^*(x)$ 位段划分明确了地址语义; 图 2-2 与 facility-cubby-Slot (槽位数组、 A 、 M 、 B) 层级关系给出整体骨架; 商压缩说明高位载荷与 $B[i]$ 的信息分工^[5,17]; 同时从 k-kick 交换上界、MiniArray M 的空闲判定、Local Query Router 的 $O(1)$ 查询及四个状态同步不变量四个角度, 为第三章算法设计提供理论锚点。下一章将给出各模块的完整伪代码; 第四章则报告宏基准与参数扫描下的性能测量。

第三章 哈希表结构与算法设计

本章说明分层哈希表从理论设定到原型实现的对应关系。内容包括 facility、MiniArray、多层级 cubby、cubby 内 k-kick、Local Query Router 与商压缩实现，并统一给出插入、查询和删除语义。

3.1 facility 与 MiniArray

3.1.1 facility 的基本属性

每个 facility 对应全局表中的一个局部片段，规模为 $K = \text{polylog } N$ 。facility 管理其内部的全部 cubby（包括各层级及唯一 tail cubby），并维护一棵 facility 层的 MiniArray，用于定位键对应的 cubby。B 树的扇出（每个内部节点的子指针个数）记为 F ，取 $F = \Theta(\log n / \log \log n)$ ；树高为 $O(1)$ ，每个内部节点的子指针仅需 $\Theta(\log \log n)$ 比特^[5,32]。

3.1.2 MiniArray 的逻辑结构

MiniArray 由常数深度 B 树、紧凑叶节点、位图、秩/选择查询和查表结构共同组成：

MiniArray $\rightarrow O(1)$ 深度 B 树 $\rightarrow$ 紧凑叶节点 + 位图 + Rank/Select + 查表

叶节点包含两部分：(1) 位图，用于标记叶内哪些逻辑位置已有有效元素，其大小与叶容量相同；(2) 紧凑数组，在有效位置上无间隙地存放变长元素。

内部节点只保留定位所需信息：(1) subtree_size[F]，记录每个子树中的有效元素总数，供 Rank/Select 定位；(2) children[F]，至多 F 个子女指针，每个指针 $\Theta(\log \log n)$ 比特。

MiniArray 对外提供的接口函数为 Access(idx)、Insert(idx, val)、Delete(idx)、Update(idx, val)，其中 idx 为逻辑下标。叶容量上限为 MAX_LEAF_SIZE，由

NODE_MAX_BITS 常数 $c \in [1, 3]$ 产生^[5,13,24]。

3.1.3 Access: 按逻辑下标读取

访问操作的输入是逻辑下标。搜索过程自根向下遍历, 利用各层 `subtree_size` 进行前缀和判断, 将逻辑下标映射到目标叶节点; 随后在叶内用 `Select` 在位图上定位第 `idx` 个有效位置, 再从紧凑数组读出元素。树高为 $O(1)$, 叶内 `Select` 通过查表常数时间完成^[21,24,34], 流程见算法 3.1。

算法 3.1 MiniArray.Access

```

1: 输入逻辑下标 idx
2: cur ← root, rem ← idx
3: while cur 为内部节点 do
4:   找到子树  $i$  使  $\sum_{t < i} subtree\_size[t] \leq rem < \sum_{t \leq i} subtree\_size[t]$ 
5:   rem ← rem -  $\sum_{t < i} subtree\_size[t]$ 
6:   cur ← cur.child[i]
7: end while
8: pos ← CallSelect(cur.bitmap, rem)
9: 返回 cur.data[pos]
```

3.1.4 Insert: 插入与叶分裂

插入过程在定位叶节点时维护路径栈, 供后续自底向上更新 `subtree_size`。到达叶节点后, 用 `Select` 找到插入位置, 将位图对应位置置为 1, 并向紧凑数组写入 `val`。若叶内有效元素数达到容量上限, 则调用 `Split` 分裂叶节点; 插入与分裂检查完成后, 再沿路径栈把各内部节点的 `subtree_size` 加一。具体流程见算法 3.2。

3.1.5 Delete: 删除与叶合并

删除过程同样沿 `subtree_size` 定位叶节点, 用 `Select` 在位图上找到待删元素位置, 清除对应位, 并从紧凑数组移除元素。若叶内有效元素低于阈值, 则调用 `Merge` 与相邻兄弟叶合并; 路径上的 `subtree_size` 随后自底向上减一。该过程见算法 3.3。

算法 3.2 MiniArray.Insert

```
1: 输入 idx, val; 初始化空路径栈 stk
2: cur ← root, rem ← idx
3: while cur 为内部节点 do
4:   stk.push(cur)
5:   找到子树 i 使  $\sum_{t < i} subtree\_size[t] \leq rem < \sum_{t \leq i} subtree\_size[t]$ 
6:   rem ← rem -  $\sum_{t < i} subtree\_size[t]$ 
7:   cur ← cur.child[i]
8: end while
9: leaf ← cur
10: pos ← CallSelect(leaf.bitmap, rem)
11: leaf.bitmap.set(pos); leaf.data.insert(pos, val)
12: if leaf.size ≥ MAX_LEAF_SIZE then
13:   CallSplit(leaf)
14: end if
15: for 路径栈中每个内部节点 node do
16:   node.subtree_size[i] += 1
17: end for
```

算法 3.3 MiniArray.Delete

```
1: 输入 idx; 初始化路径栈 stk
2: 沿 subtree_size 定位至叶 leaf(同 Insert)
3: pos ← CallSelect(leaf.bitmap, rem)
4: leaf.bitmap.unset(pos); leaf.data.remove(pos)
5: if leaf 有效元素过少 (即 leaf.size < MAX_LEAF_SIZE / 2) then
6:   CallMerge(leaf)
7: end if
8: for 路径栈中每个内部节点 node do
9:   node.subtree_size[i] -= 1
10: end for
```

3.1.6 Update: 原地覆写

更新过程仅修改已有有效位置的元素值，不改变位图中 1 的个数，也不触发 Split/Merge。定位过程与访问操作相同，在叶内 Select 后直接令 $\text{leaf.data}[\text{pos}] \leftarrow \text{val}$ ，如算法 3.4 所示。

算法 3.4 MiniArray.Update

- 1: 输入 idx, val
 - 2: 沿 subtree_size 定位至叶 leaf
 - 3: $\text{pos} \leftarrow \text{CallSelect}(\text{leaf.bitmap}, \text{rem})$
 - 4: $\text{leaf.data}[\text{pos}] \leftarrow \text{val}$
-

3.1.7 Split 与 Merge

Split（叶分裂）。新建叶节点，以 $\text{leaf.size}/2$ 为界将后半段位图与紧凑数组迁至新叶，原叶保留前半段；随后在父节点 child 数组中插入新叶指针，并更新 subtree_size ，流程见算法 3.5。

算法 3.5 MiniArray.Split（叶分裂）

- 1: 输入叶 leaf
 - 2: $\text{new_leaf} \leftarrow$ 新建 LeafNode ; $\text{mid} \leftarrow \text{leaf.size}/2$
 - 3: 将 leaf.bitmap 、 leaf.data 后半段搬至 new_leaf
 - 4: leaf 保留前半段
 - 5: 在父节点 parent.child 中插入 new_leaf ，更新 $\text{parent.subtree_size}$
-

Merge（叶合并）。取相邻兄弟叶，将兄弟的位图与紧凑数组并入当前叶，从父节点删去兄弟子树并刷新 subtree_size ，流程见算法 3.6。

算法 3.6 MiniArray.Merge（叶合并）

- 1: 输入叶 leaf ; $\text{sib} \leftarrow$ 相邻兄弟叶; $\text{parent} \leftarrow \text{leaf}$ 的父节点
 - 2: $\text{leaf.bitmap} \mid = \text{sib.bitmap}$; $\text{leaf.data.append}(\text{sib.data})$
 - 3: 从 parent.child 移除 sib ，更新 $\text{parent.subtree_size}$
-

3.1.8 Rank 与 Select

叶子节点中位图上的 Rank/Select 通过查表实现，这是紧凑动态结构的标准组件^[5,12,21,34]。Rank 返回位图前 i 位中 1 的个数，Select 返回位图中第 k 个 1 的位置；两者的伪代码见算法 3.7。

算法 3.7 Rank / Select（查表实现）

```
1: function RANK(bmp, i)
2:   返回 LookupTable[bmp 高段] + LookupTable[bmp 低段]
3: end function
4: function SELECT(bmp, k)
5:   返回 LookupTable[k] ▷ 第  $k$  个 1 的位置
6: end function
```

3.1.9 复杂度与空间性质

facility 级 MiniArray 的树高为 $O(1)$, 自根至叶的每层路由仅依赖 subtree_size 比较, 叶内 Rank/Select 经查表结构常数时间完成^[21,34]。因此 Access 与 Update 为最坏情况 $O(1)$; Insert 与 Delete 在叶分裂/合并均摊意义下亦为 $O(1)$ ^[5,32]。空间方面, 每个内部节点的 F 个子指针各需 $\Theta(\log \log n)$ 比特, 叶节点以位图标记有效位置、以紧凑数组无间隙存放变长元素, 整体辅助位数与 $O(\log^{(k)} n)$ 的全局预算相容^[13,24]。cubby 内的 A 、 M 、 B 三组 MiniArray 复用同一套接口语义: $A[i]$ 存局部路由映射, M 存子区间空闲图, B 与槽位数组一一对应存差分编码。三者在 k-kick 搬移时须同步更新。

3.2 多层级 cubby 设计

3.2.1 层级属性与初始状态

facility 内 cubby 按层级管理。第 j 层 cubby 的目标数量为

$$t_j = \Theta\left(\frac{(\log^{(j+1)} n)^2}{(\log^{(j)} n)^2}\right), \quad (3-1)$$

容量（槽位数）为

$$r_j = \frac{K}{(\log^{(j)} n)^2}. \quad (3-2)$$

系统初始时, 每个 facility 仅包含一个第 1 层 cubby, 且该 cubby 即为 tail cubby。此后所有新键插入 tail cubby; tail cubby 满后, 旧 tail cubby 被视为一个普通第 1 层 cubby 挂入层级集合, 并新建 tail cubby 承接后续插入, 该过程可能触发层级向上合并^[1,5,25-26]。

3.2.2 动态调整：合并与拆分

设当前第 j 层 cubby 的实际个数为 cnt_j ，调度规则见算法 3.8。(1) 当 $\text{cnt}_j \geq 3t_j$ 时，取出 t_j 个第 j 层 cubby，将其中元素导出，合并为一个第 $j+1$ 层 cubby 后重新插入；(2) 当 $\text{cnt}_j \leq t_j/2$ 且存在非空的第 $j+1$ 层 cubby 时，取出一个第 $j+1$ 层 cubby，拆成 t_j 个第 j 层 cubby 并重插元素；(3) 无论合并或拆分，均须取出 cubby 内所有元素重新插入，并同步更新对应 cubby 中的 MiniArray A 、 M 、 B 与槽位数组，不能仅复制旧槽位内容，因为 r_j 、 $|I|$ 与商压缩上下文可能已经变化^[5,8,20]。

算法 3.8 第 j 层 cubby 数量调度

```

1: for 每个层级  $j$  do
2:   计算目标数量  $t_j$ ，统计实际数量  $\text{cnt}_j$ 
3:   if  $\text{cnt}_j \geq 3t_j$  then
4:     取出  $t_j$  个第  $j$  层 cubby，合并为一个第  $j+1$  层 cubby 并重新插入
5:   else if  $\text{cnt}_j \leq t_j/2$  且第  $j+1$  层非空 then
6:     取出一个第  $j+1$  层 cubby，拆成  $t_j$  个第  $j$  层 cubby 并重新插入
7:   end if
8: end for

```

3.2.3 cubby 内部组件

每个容量为 $|I|$ 的 cubby 包含五部分：

(1) 槽位数组。长度为 $|I|$ ，存放键值的商压缩载荷；槽位 i 上的内容对应 $\pi(x)[\log N + 1, \log U]$ 的高位载荷部分，完整键恢复依赖 $B[i]$ 与全局上下文。

(2) k-kick 插入逻辑。在槽位数组上按固定规则决定元素实际槽位，并维护每个占用元素的 `insert_depth`。

(3) MiniArray A 。长度为 $|I|$ 的 Local Query Router 数组； $A[i]$ 是一棵二叉前缀树，管理所有满足 $g_I(x) = i$ 的键。

(4) MiniArray M 。存储各 k-kick 深度子区间的空闲槽位图，使插入、删除与 ReInsert 能在常数时间内判定子区间是否含空位、定位空位，而不必扫描整个 cubby。

(5) MiniArray B 。与槽位数组一一对应。当槽位 i 存放键 x 的信息时， $B[i]$

保存

$$\delta = g_I(x) - i, \quad (3-3)$$

以及 $g^*(x)[\log |I|+1, \log K]$ 的中间位。配合 facility 编号 $r = g^*(x)[\log K+1, \log N]$ 与高位载荷，可完整恢复 $\pi(x)$ ^[5,21]。

3.2.4 插入与删除操作

插入。每次插入均进入当前 facility 的 tail cubby。若 tail cubby 仍有空位，则在其中执行 k-kick 插入；若 tail cubby 已满，则新建 tail cubby，并将已满的旧 tail cubby 视作第 1 层 cubby 挂入第 1 层集合，该步骤可能因 $\text{cnt}_1 \geq 3t_1$ 而触发向上合并。

删除与 tail cubby 回填。若删除发生在 tail cubby 之外的其他 cubby，删除完成后从 tail cubby 中随机选取一个占用元素，经 k-kick 规则回填至刚空出的槽位，并更新 MiniArray A 、 M 、 B 与槽位数组。若 tail cubby 在回填后为空，则从第 1 层取出一个 cubby 提升为新的 tail cubby；该过程可能因 $\text{cnt}_i \leq t_i$ 而触发向下拆分，甚至触发多层调整。上述不变量使非 tail cubby 尽量维持满载，局部波动由 tail cubby 与层级调度共同吸收，伪代码见算法 3.9^[5,22-23]。

算法 3.9 tail cubby 维护与删除回填

- 1: 插入：写入 tail cubby；若 tail cubby 满则旧 tail cubby 归入第 1 层 cubby 并新建 tail cubby
 - 2: 删除（非 tail cubby）：释放被删槽位；从 tail cubby 随机取元素 y 回填
 - 3: 对 y 执行 k-kick 写入，更新 A, M, B
 - 4: **if** tail cubby 为空 **then**
 - 5: 从第 1 层 cubby 提升一个 cubby 为 tail cubby；必要时触发层级拆分
 - 6: **end if**
-

3.3 k-kick 层级插入算法

约定：cubby 内算法中的 x 均指 $\pi(x)$ （置换后的值），而非原始键；符号仍记为 x ，设 cubby 容量为 $|I|$ 。

3.3.1 层级子区间定义

k-kick 不对槽位数组做物理分层，仅在逻辑上将 cubby 划分为 $k + 1$ 个深度， k 为常数（通常 $k \in [3, 5]$ ，基准取 4）。深度 0 的唯一子区间覆盖整个 cubby，大小 $s_0 = K = \text{polylog } n$ ；深度 1 将深度 0 的子区间均匀切分为许多大小 $s_1 = \Theta((\log K)^6)$ 的子区间；一般地，深度 i ($i \geq 1$) 的子区间大小为 $s_i = \Theta((\log^{(i)} K)^6)$ ，由上一层子区间继续均匀切分得到。

定义函数 $\text{Bin}(x, i)$ 返回键 x 在深度 i 所属的槽位子区间：深度 0 时返回 $[0, |I| - 1]$ ；深度 $i \geq 1$ 时在 $\text{Bin}(x, i - 1)$ 内按 s_i 均匀切分，并根据 x 的哈希比特在每一层中选定对应的一个子区间^[5,31]。

3.3.2 偏好序列

对键 x 定义偏好序列，构造过程见算法 3.10：

$$g_0(x) \rightarrow g_1(x) \rightarrow \cdots \rightarrow g_k(x). \quad (3-4)$$

其中 $g_0(x) = \text{Bin}(x, 0)$ 为整个 cubby； $g_{i+1}(x)$ 为 $g_i(x)$ 经均匀切分后 x 所落子区间。于是 $g_{i+1}(x)$ 一定是 $g_i(x)$ 的子区间；沿路径自顶向下，每一层均为上一层的随机子区间。自底向上回溯 $g_k, g_{k-1}, \dots, g_0$ 即可得到全部祖先子区间。

算法 3.10 PreferenceSequence: 偏好序列

```

1: function PREFERENCESEQUENCE( $x$ )
2:    $g[0] \leftarrow \text{BIN}(x, 0)$ 
3:   for  $i = 1$  to  $k$  do
4:      $g[i] \leftarrow g[i - 1]$  内均匀切分后、 $x$  所属的子区间
5:   end for
6:   返回  $g[0..k]$ 
7: end function

```

3.3.3 探测位置序列

$h_j(x)$ 表示 x 在 cubby 槽位数组中的第 j 个候选槽位。序列按先深后浅顺序构造：依次遍历 $g_k(x), g_{k-1}(x), \dots, g_0(x)$ ，并在每个子区间内按槽位顺序枚举全部位置，伪代码见算法 3.11。

对深度 i 的子区间 $g_i(x)$, 设

$$\text{base}_i = s_k + s_{k-1} + \cdots + s_{i+1}, \quad (3-5)$$

则对任意 $j \in [0, s_i - 1]$, 全局探测序列中对应项为

$$h_{\text{base}_i+j}(x) = g_i(x) \text{ 的第 } (j+1) \text{ 个槽位}. \quad (3-6)$$

注意文档中下标公式 $h_{(k+1-i) \cdot s_i + j}(x)$ 与 base_i 表述等价: 更深层的子区间在序列中排在更前。序列总长度约为 $\sum_{i=0}^k s_i$; 查询阶段并不线性扫描该序列, 而由 Local Query Router 直接给出实际使用的下标 $j^{[5-6]}$ 。

算法 3.11 ProbeSequence: 探测位置序列 (先深后浅)

```

1: function PROBESEQUENCE( $x$ )
2:    $g \leftarrow \text{PREFERENCESEQUENCE}(x)$ ;  $seq \leftarrow []$ 
3:   for  $i = k$  downto 0 do
4:      $bin \leftarrow g[i]$ 
5:     for  $pos$  从  $bin.start$  到  $bin.end$  do
6:        $seq.append(pos)$ 
7:     end for
8:   end for
9:   返回  $seq$ 
10: end function

```

$\triangleright h_1(x), h_2(x), \dots$ 依次为 $seq[0], seq[1], \dots$

3.3.4 子区间饱和与最大可用深度

在深度 d 下, 称子区间饱和, 当且仅当该子区间已满, 且其中每个元素 y 均满足 $y.insert_depth \geq d$ 。子区间未满足时不饱和; 子区间已满但存在 $y.insert_depth < d$ 的元素时, 亦不视为饱和。

GetMaxUsableDepth(x) 自 k 向 0 扫描偏好序列, 返回最深的饱和子区间的深度; 若全部饱和则返回 0。饱和判定与扫描过程见算法 3.12。

算法 3.12 IsSaturated 与 GetMaxUsableDepth

```
1: function ISSATURATED(bin,  $d$ )
2:   if bin 未滿 then
3:     返回 false
4:   end if
5:   for bin 中每个元素  $y$  do
6:     if  $y.insert\_depth < d$  then
7:       返回 false
8:     end if
9:   end for
10:  返回 true
11: end function
12: function GETMAXUSABLEDEPTH( $x$ )
13:   $g \leftarrow \text{PREFERENCESEQUENCE}(x)$ 
14:  for  $d = k$  downto 0 do
15:    if not ISSATURATED( $g[d]$ ,  $d$ ) then
16:      返回  $d$ 
17:    end if
18:  end for
19:  返回 0
20: end function
```

3.3.5 随机深度选择与 InsertKick

插入时须记录 $insert_depth$ 并在各深度子区间均匀分布。算法 3.13 中先取随机哈希 $s(x) \in [0, k]$, 再令

$$depth = \min(\text{GETMAXUSABLEDEPTH}(x), s(x)). \quad (3-7)$$

该式为随机深度与可用深度的 min 组合, 对贪心上界形成软上限, 使高深度子区间更长时间保持未饱和。在目标子区间 $g_{depth}(x)$ 中: 若有空位, 则写入 x 、记录 $x.insert_depth = depth$, 并在 Local Query Router $A[g_I(x)]$ 中登记探测下标 j ; 若无空位, 则踢出子区间内 $insert_depth$ 最小的元素 y , 将 x 写入被踢槽位, 再对 y 调用 ReInsert。整条踢出链至多触发 k 次, 与 $O(k)$ 交换上界一致^[5]。

3.3.6 ReInsert: 被踢元素向上放入

插入发生踢出时, 被踢元素须安置到合适深度。ReInsert 自 $y.insert_depth-1$ 向上逐层检查 $\text{Bin}(y, d)$: 若该子区间在深度 d 下不饱和且有空位, 或存在 $insert_depth$ 小于 d 的元素, 则将 y 写入并可能继续踢出链式调用 ReInsert。深度降

算法 3.13 InsertKick: k-kick 插入

```
1: 输入  $x$  (已为  $\pi(x)$ )
2:  $depth\_max \leftarrow \text{GETMAXUSABLEDEPTH}(x)$ 
3:  $s_x \leftarrow \text{RANDOMHASH}(x) \bmod (k + 1)$ 
4:  $depth \leftarrow \min(depth\_max, s_x)$ 
5:  $g \leftarrow \text{PREFERENCESEQUENCE}(x)$ ;  $target \leftarrow g[depth]$ 
6: if  $target$  有空位 (由  $M$  判定) then
7:   在任意空位写入  $x$ ;  $x.insert\_depth \leftarrow depth$ 
8:   更新  $A, M, B$ ; 返回成功
9: else
10:   $y \leftarrow target$  中  $insert\_depth$  最小元素;  $evict\_pos \leftarrow y$  的位置
11:  从  $evict\_pos$  移除  $y$ ; 将  $x$  写入  $evict\_pos$ 
12:   $x.insert\_depth \leftarrow depth$ 
13:   $\text{REINSERT}(y)$ 
14: end if
```

至 0 仍无法安置时, cubby 已满, 须新建 tail cubby 并重试 InsertKick。MiniArray M 在此过程中的空位判定与定位均为常数时间。理论保证: 在参数满足 Bender 等人^[5] 的设定时, 极大概率至多 k 次踢出后一定能安置被踢元素。该过程见算法 3.14。

算法 3.14 ReInsert: 被踢元素向上找空位

```
1: function  $\text{REINSERT}(y)$ 
2:   for  $d = y.insert\_depth - 1$  downto 0 do
3:      $bin \leftarrow \text{BIN}(y, d)$ 
4:     if  $\text{ISATURATED}(bin, d)$  then
5:       continue
6:     end if
7:     if  $bin$  有空位 (由  $M$  判定) then
8:       将  $y$  写入空位;  $y.insert\_depth \leftarrow d$ 
9:       更新  $A, M, B$ ; 返回成功
10:    else if  $bin$  中存在  $z.insert\_depth < d$  then
11:       $z \leftarrow bin$  中  $insert\_depth$  最小且满足  $z.insert\_depth < d$  的元素
12:      从  $bin$  移除  $z$ ; 将  $y$  写入该槽位
13:       $y.insert\_depth \leftarrow d$ ; 更新  $A, M, B$ 
14:       $\text{REINSERT}(z)$ ; 返回
15:    end if
16:  end for
17:  返回失败 ▷ 深度已降至 0 仍无法安置, cubby 已满
18:  新建 tail cubby; 在新 tail cubby 中对  $y$  重试  $\text{INSERTKICK}$ 
19: end function
```

3.3.7 复杂度与正确性

单次 InsertKick 至多触发一条长度为 k 的踢出链：每次踢出仅调用一次 ReInsert，而被踢元素的 insert_depth 严格下降，故链长受 k 约束，与 Bender 等人^[5]的 $O(k)$ 交换成本一致。

GetMaxUsableDepth 自 k 向 0 扫描，返回最深的饱和子区间；随机深度 $\text{depth} = \min(\text{depth_max}, s(x))$ 在理论上保证：若存在可写子区间，则插入不会触发深层踢出，同时使不同深度的子区间负载趋于均衡。

子区间中是否存在空位与定位空位都通过 MiniArray M 完成，避免对整个 cubby 进行依次遍历 $O(|I|)$ 时间复杂度的扫描；ReInsert 从 $y.\text{insert_depth}-1$ 向上逐层检查，每层仅访问 M 与局部子区间状态，均摊常数时间。

元素每次写入或迁移，都需要同步更新槽位数组、insert_depth、 B 、 $g_I(x)$ 中的 $\pi(x) \mapsto j$ 映射对应的信息，以及 M 的空闲位图，这一同步规则是整个系统正常运行的前提。

3.4 Local Query Router

3.4.1 设计目标

偏好序列与探测位置序列 $h_1(x), h_2(x), \dots$ 在插入分析中完整定义了候选槽位顺序。然而 k-kick 插入会通过踢出动态挪动元素：元素实际位置一般不等于首选深度 $g_k(x)$ ，也不等于 $h_1(x)$ 。若查询仍沿 $h_1, h_2, \dots$ 线性扫描，时间复杂度将与探测序列长度同阶，无法达到 $O(1)$ 。因此必须为每个键保存精准映射

$$\pi(x) \longrightarrow j, \quad (3-8)$$

其中 j 为探测序列中的下标，查询时直接访问下标 $h_j(x)$ 对应的槽位，再用 B 与高位载荷恢复 $\pi(x)$ 后与查询元素进行对比校验^[5,21]。

3.4.2 映射规则

cubby 内定义局部首选槽位号（与 k-kick 偏好序列无关）：

$$g_I(x) = g^*(x)[1, \log |I|] \in [0, |I| - 1]. \quad (3-9)$$

所有满足 $g_I(x) = i$ 的键由第 i 号 Local Query Router $A[i]$ 管理。查询时先计算 $i = g_I(x)$ ，再在 $A[i]$ 中查询 $\pi(x)$ 得到探测序列下标 j （若不存在则未命中）；随后访问 $h_j(x)$ 对应槽位，读取高位载荷与 B ，重建 $g^*(x)$ 并校验是否命中。

3.4.3 二叉前缀树存储与接口

每个 $A[i]$ 实现为一棵二叉前缀树，对外键为 $\pi(x)$ 的比特串。接口包括 `insert(key, j)`、`query(key)` 和 `delete(key)`：insert 记录 $j \leq \log K$ 的探测下标，每条映射只需 $O(\log \log n)$ 比特；query 返回 j 或“不存在”；delete 删除对应映射。三者分别见算法 3.15、算法 3.16 和算法 3.17。

树中不存完整键，仅存区分键的最短前缀；叶节点存 `j_value`。

插入过程利用映射后的 $\pi(x)$ 进行操作；下文以 x 表示 $\pi(x)$ 的比特串。映射后的 x 的每一位（从高位到低位）依次决定在树中向左（0）还是向右（1）走。沿路径向下，若遇空子树则创建新节点；到达叶节点时标记为叶并存 `j_value`。即使管理的键数较多时，树高仍受限于键的位数，故访问步数为 $O(\log U)$ 。

算法 3.15 Local Query Router.insert

```

1: function INSERT(key, j)
2:   cur  $\leftarrow$  root
3:   for key 的每一位 bit do
4:     if bit = 0 then
5:       if cur.child0 为空 then 创建子节点
6:       end if
7:       cur  $\leftarrow$  cur.child0
8:     else
9:       if cur.child1 为空 then 创建子节点
10:      end if
11:      cur  $\leftarrow$  cur.child1
12:    end if
13:  end for
14:  cur.is_leaf  $\leftarrow$  true; cur.j_value  $\leftarrow$  j
15: end function

```

算法 3.16 Local Query Router.query

```
1: function QUERY(key)
2:   cur  $\leftarrow$  root
3:   for key 的每一位 bit do
4:     if bit = 0 then
5:       if cur.child0 为空 then 返回 NOT_FOUND
6:       end if
7:       cur  $\leftarrow$  cur.child0
8:     else
9:       if cur.child1 为空 then 返回 NOT_FOUND
10:      end if
11:      cur  $\leftarrow$  cur.child1
12:    end if
13:  end for
14:  if cur.is_leaf then 返回 cur.j_value
15:  else 返回 NOT_FOUND
16:  end if
17: end function
```

算法 3.17 Local Query Router.delete

```
1: function DELETE(key)
2:   沿 key 的比特路径找到叶节点
3:   将叶标记为非叶 (is_leaf  $\leftarrow$  false), 清除 j_value
4: end function
```

3.4.4 复杂度与 Patricia 压缩

对键长 $\log U = \Theta(\log n)$ 的比特串, query 与 insert 沿树向下至多 $\log U$ 步; Patricia 风格的前缀压缩^[17,21,36] 将仅有一位不同的键合并为同一压缩边, 使实测路由深度 (router_steps_max) 远小于键长上界。每条映射存 $j \leq \log K = O(\log \log n)$ 比特, 与整体 $O(\log^{(k)} n)$ 空间目标一致^[5,20]。

动态维护。k-kick 踢出链中, 被搬移键须删除旧映射再插入新 j ; 删除操作清空叶 j_value 但不破坏共享前缀路径上的内部节点, 均摊开销与 $A[i]$ 所管理的局部键数相关。在 $K = \text{polylog } n$ 且 cubby 容量 $|I| \leq K$ 的设定下, 单个 $A[i]$ 的规模受控, 路由更新与查询均摊为常数时间^[1,5]。

3.5 商压缩与键恢复

3.5.1 设计动机与信息分工

当前层次结构中, 查询路由需用映射后二进制低位定位 facility 与 cubby 内数组; 若槽位仍存完整键, 则寻址路径上的信息上下文被重复占用。商压缩借助随机可逆置换 π , 将键映射为 $\pi(x)$, 并按下述原则拆分信息: $\pi(x)$ 的低 $\log N$ 位 $g^*(x) = \pi(x)[1, \log N]$ 的各段位由全局路由与局部地址上下文 (facility 编号、cubby 容量、实际槽位下标) 及 MiniArray B 中的差分元数据共同承担; 槽位数组仅保存无法由上述上下文直接推出的高位商, 即高位载荷

$$\text{payload} = \pi(x)[\log N + 1, \log U]. \quad (3-10)$$

查询时 Local Query Router 给出探测位置序列下标 j , $h_j(x)$ 定位到槽位数组中的具体位置, 然后从该槽位、 $B[h_j(x)]$ 与 facility/cubby 上下文联立恢复完整 $\pi(x)$, 再经 π^{-1} 得到原始键。该分工使每个元素在槽位层面仅占用 $\log U - \log N$ 比特的有效载荷, 而将路由相关的 $\log N$ 位分散到地址结构与变长元数据中, 与整体 $O(\log^{(k)} n)$ 空间目标一致^[5,21]。

3.5.2 $\pi(x)$ 与 $g^*(x)$ 的位段划分

记 $x[a, b]$ 为 x 自第 a 位至第 b 位（自低位向高位）组成的子串。对键 x 经置换得 $\pi(x)$ ，全局路由哈希为

$$g^*(x) = \pi(x)[1, \log N]. \quad (3-11)$$

在 $g^*(x)$ 内部，自高向低继续划分为三段（见图 2-1，前文已给出示意）：(1) facility 编号段 $r = g^*(x)[\log K + 1, \log N] = \lfloor g^*(x)/K \rfloor$ ，用于定位键所属 facility；(2) cubby 中间段 $g^*(x)[\log |I| + 1, \log K]$ ，用于在 facility 内区分不同 cubby 及局部桶范围；(3) Local Query Router 段 $g_I(x) = g^*(x)[1, \log |I|] \in [0, |I| - 1]$ ，与 k-kick 偏好序列无关，仅决定键归属 $A[i]$ 的桶号 i 。

k-kick 插入可能将元素 x 写入槽位 $i \neq g_I(x)$ 的实际位置。此时 $g_I(x)$ 无法由槽位下标 i 直接读出，须额外保存差分

$$\delta = g_I(x) - i, \quad (3-12)$$

存入 MiniArray $B[i]$ 。

3.5.3 MiniArray B 中的 MetaEntry

每个占用槽位 i 对应一条变长元数据 MetaEntry，经 MiniArray B 的 Access/Update 接口读写。逻辑字段包括两部分：(1) delta，即有符号整数 $\delta = g_I(x) - i$ ，编码为 zigzag varint，用于适应 k-kick 搬移后槽位与路由桶号的不一致；(2) mid_bits，即 $g^*(x)$ 在 cubby 范围内的中间位。令 $g_k = g^*(x) \bmod K$ ，则 $g_k = \text{mid_bits} \cdot |I| + g_I(x)$ ，其中 mid_bits 对应商部分 $\lfloor g_k / |I| \rfloor$ ，位宽 $\text{mid_w} = \lceil \log_2((K - 1) / |I| + 1) \rceil$ ，由 $(K, |I|)$ 派生。

编码顺序为：先写入 delta 的 varint，再写入 mid_bits 的固定位宽字段；解码时按相同布局与 MetaCodecLayout 逆序读出。 $B[i]$ 的 bitlen 随条目实际长度变化，由 facility 级 MiniArray 的 Rank/Select 机制定位，与前文所述变长元数据维护方式一致。

3.5.4 恢复：重建 $\pi(x)$

给定 facility 编号 r 、cubby 容量 $|I|$ 、槽位下标 i 、槽位中的高位载荷及 $B[i]$ 解码得到的 MetaEntry，可按算法 3.18 自底向上重建 $g^*(x)$ 与 $\pi(x)$ ：

$$g_I(x) = i + \delta, \quad (3-13)$$

$$g_k = \text{mid_bits} \cdot |I| + g_I(x), \quad (3-14)$$

$$g^*(x) = r \cdot K + g_k, \quad (3-15)$$

$$\pi(x) = (\text{payload} \ll \log N) \mid g^*(x). \quad (3-16)$$

各步均须校验 $g_I(x) \in [0, |I|)$ 、 $g_k < K$ 及 $g^*(x) < N$ ；任一不满足则恢复失败，视为未命中。原始键由 $x = \pi^{-1}(\pi(x))$ 得到。

算法 3.18 ReconstructGxPi: 从槽位上下文恢复 $\pi(x)$

```

1: function RECONSTRUCTGxPi( $r, N, K, |I|, \text{slot}, \text{payload}, \text{meta}, \log N$ )
2:    $g_I \leftarrow \text{slot} + \text{meta}.\text{delta}$ 
3:   if  $g_I \notin [0, |I|)$  then
4:     返回失败
5:   end if
6:    $g_k \leftarrow \text{meta}.\text{mid\_bits} \cdot |I| + g_I$ 
7:   if  $g_k \geq K$  then
8:     返回失败
9:   end if
10:   $g_{\text{star}} \leftarrow r \cdot K + g_k$ 
11:  if  $g_{\text{star}} \geq N$  then
12:    返回失败
13:  end if
14:  返回  $(\text{payload} \ll \log N) \mid g_{\text{star}}$ 
15: end function

```

3.5.5 查询校验

查询键 x 时，Router 给出探测下标 j ，访问候选槽位 $i = h_j(x)$ 。只有当完整恢复的值与查询的值相同，才认为查询命中，校验过程见算法 3.19：

路由器仅负责将查询映射到 $O(I)$ 个候选位置；最终是否命中必须由高位载荷、 B 与 $(r, N, K, |I|)$ 联立恢复的 $\pi(x)$ 与查询侧计算的 $\pi(x)$ 一致方可确认。该“路由器定位 + 商压缩校验”的两段式路径，是系统能够不存储完整键的前

算法 3.19 SlotPiMatches: 槽位命中校验

```
1: function SLOTPIMATCHES(cubby, slot,  $r$ , table, query_gx)
2:   if cubby.slots[slot] 为空 then
3:     返回 false
4:   end if
5:   meta  $\leftarrow$  DECODEMETAENTRY(cubby.B[slot])
6:   gx  $\leftarrow$  RECONSTRUCTGXPI( $r$ ,  $N$ ,  $K$ ,  $|I|$ , slot, payload, meta, logN)
7:   if gx 失败 then
8:     返回 false
9:   end if
10:  返回 gx = query_gx
11: end function
```

提^[5,21]。

3.6 插入、查询与删除

插入。计算 $g^*(x) = \pi(x)[1, \log N]$ 与 facility 编号 r ；在对应 facility 的 tail cubby 内执行 InsertKick（算法 3.13），写入槽位数组、更新 M 与 $B[i]$ ，并在 $A[g_I(x)]$ 中插入 $\pi(x) \mapsto j$ 映射。tail cubby 满则挂入第 1 层 cubby 并新建 tail cubby，必要时按算法 3.8 触发合并^[5]。

查询。计算 $g^*(x)$ 定位 facility 与 cubby；令 $i = g_I(x)$ ，在 $A[i]$ 中查询 $\pi(x)$ 得 j ；访问 $h_j(x)$ ，用高位载荷、 B 、 r 重建 $g^*(x)$ 并与 $\pi(x)$ 比较。该路径仅含常数次 MiniArray 与 Local Query Router 访问，查询时间为 $O(1)$ ^[5]。

删除。定位元素所在 cubby 与槽位，清空槽位数组中对应位置，更新 $A[g_I(x)]$ 、 M 与 B 。若删除发生在非 tail cubby，按算法 3.9 从 tail cubby 随机抽取元素回填；回填写入同样走 k-kick 与 Local Query Router 更新路径，以恢复各子区间的饱和与深度不变量。

表级扩缩容时， N 、 K 、 r_j 与商压缩上下文变化，须先恢复原始键再按新参数重插，而不能直接复制旧槽位的高位载荷与 B ^[1,5]。

3.7 冷热分层键存储实现

原始多层次小桶结构侧重紧凑空间和常数时间查询，所有键都由内存中的 facility、cubby 与 Slot 管理。该结构适合观察 k-kick、MiniArray 和 Local Query

Router 的组合效果，但在工程实现中也带来一个问题：冷数据仍然要参与层级合并、拆分和删除回填。若进一步要求持久化，直接把每一层 cubby 的状态写入磁盘并不方便。cubby 容量、槽位下标和 B 中的差分信息依赖当前结构上下文，Local Query Router 的映射也会随着搬移变化。本文的冷热分层方案没有沿用完整多层级冷数据结构，而是保留 tail cubby 作为内存热层，将不再活跃的键批量下沉到 SQLite 中。

冷热分层结构仍以 facility 编号作为表级分区依据。每个分区维护一个内存 tail cubby 和一组冷数据记录。tail cubby 接收近期插入的键。其内部仍使用 k-kick 插入，并用 MiniArray M 、MiniArray B 和 Local Query Router 维护状态。SQLite 冷层只保存已经下沉的键。热层命中时，系统继续复用第三章前文的局部查询与键恢复逻辑；热层未命中时，再把冷数据当作数据库记录查询或删除。

冷层记录至少包含原始键、所属 facility 编号和必要的状态标记。实验原型中只存键，不存完整 cubby 上下文；这是一个有意的简化。原因在于下沉后的键不再参与内存 cubby 的 k-kick 置换，也不需要依赖 $g_I(x)$ 、槽位下标和 $B[i]$ 恢复键。换言之，冷热分层结构把冷数据管理从“维护多层小桶结构”改成“维护数据库键记录”。这会放弃部分紧凑空间性质，但可以减少冷数据继续参与内存层级调度带来的实现复杂度。

插入操作根据 $g^*(x)$ 定位 facility，然后把键写入该 facility 的 tail cubby。若 tail cubby 未滿，插入路径与前文 InsertKick 相同。系统先选择可用深度并写入槽位数组。随后更新 M 与 B ，并把路由映射写入 $A[g_I(x)]$ 。若 tail cubby 写滿，冷热分层结构不再把旧 tail cubby 挂入多层级 cubby 集合，而是将旧 tail cubby 中仍有效的键导出，使用 SQLite 批量事务写入冷层，随后清空内存热层并新建 tail cubby。

批量下沉保留原有哈希语义，同时减少每次插入都访问 SQLite 的成本。若每个键插入时都立即写入数据库，写入路径会频繁触发事务和索引维护；tail cubby 则把近期写入暂存在内存中，等容量达到阈值后再集中刷入数据库。本文实验中的冷热分层结构按这一口径统计插入延迟：单次插入包含热层写入和必要的轮换准备，但收尾阶段的数据库刷盘检查不计入每个操作的平均延迟。

查询操作先访问内存热层。系统根据 $g^*(x)$ 定位 facility，再计算 $g_I(x)$ ，通过 Local Query Router 查询候选槽位下标 j 。若路由器返回有效映射，则访问 $h_j(x)$

对应槽位，并通过高位载荷、 B 和上下文信息恢复 $\pi(x)$ 进行校验；校验通过时直接返回命中。若热层未命中，系统再以原始键访问 SQLite 冷层。在写后读和近期访问负载下，一部分查询会停留在内存路径；在随机冷查询或热层命中率较低时，查询仍需要进入数据库，并额外承担一次热层判断的成本。这也是第五章没有把冷热分层结构描述为无条件更快的原因。

删除操作也沿用这一顺序。如果目标键仍在 tail cubby 中，系统先删除槽位中的载荷，再清理 $A[g_I(x)]$ 中的映射，并同步更新 M 和 B 。由于冷热分层结构不把旧 tail cubby 挂入多层级 cubby 集合，删除近期键时不需要从 tail cubby 抽取元素去回填其他层级 cubby，也不会触发多层级向上合并或向下拆分。

若热层未命中，系统转向 SQLite 冷层删除对应记录。为了避免同一个键同时存在于热层和冷层，原型采用单向下沉规则：键插入后先处于热层，tail cubby 轮换时才进入冷层；冷层键不会重新搬回热层。查询和删除都按“先热层、后冷层”的顺序执行，因此即使某次下沉刚完成，后续操作也能在两层之一找到唯一记录。该规则比完整缓存淘汰策略简单，但足以支撑第五章的写后读、近期查询和近期删除实验。

和原始结构相比，冷热分层实现保留了 facility 分区、tail cubby 内的 k-kick 置换、MiniArray 元数据和 Local Query Router；热层命中时仍然通过商压缩恢复和校验键。被简化的是冷数据部分：冷数据不再组织为多层级 cubby，删除冷数据时也不再执行 tail cubby 回填和层级重构。这个边界需要明确说明：冷热分层结构是在基础原型上增加 SQLite 冷数据层，没有完整持久化 Bender 等人的理论结构。

3.8 本章小结

本章给出分层哈希表的主要结构与算法。facility 级 MiniArray 负责变长元数据定位，依赖 B 树、Rank/Select 与 Split/Merge 维持常数深度访问^[32,34]。多层级 cubby 与 tail cubby 机制吸收插入和删除带来的局部波动；k-kick 通过偏好序列和饱和判定控制单次更新中的交换次数^[5,9]。查询路径由 Local Query Router 接管，它把探测序列扫描改为对 $A[i]$ 前缀树的定位^[21,36]。这些模块不能独立更新。槽位数组、 A 、 M 、 B 在插入、踢出、删除回填和扩缩容重插中都必须保持一致。

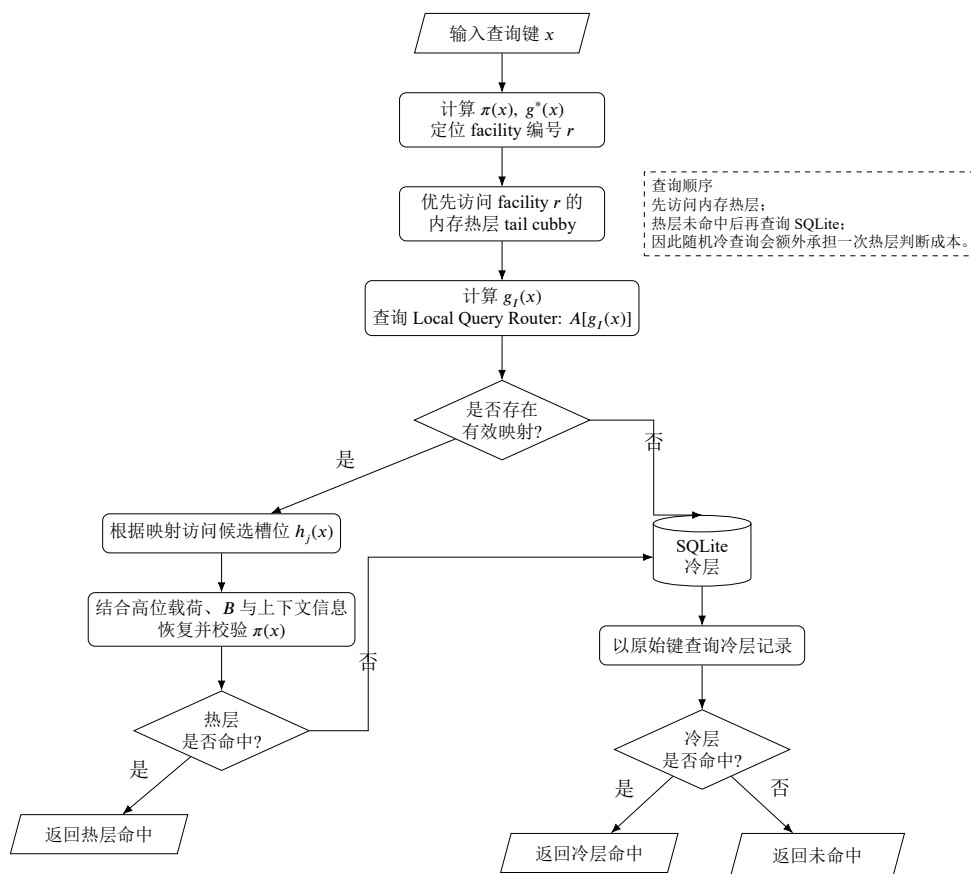


图 3-1 冷热分层结构的查询流程

冷热分层键存储在此基础上保留内存 tail cubby 的 k-kick 管理，把下沉后的键交给 SQLite 冷层保存。第四章先报告多层级原型结构在不同参数下的功能与性能测量；第五章再讨论冷热分层结构与两个对照方案之间的实验差异。

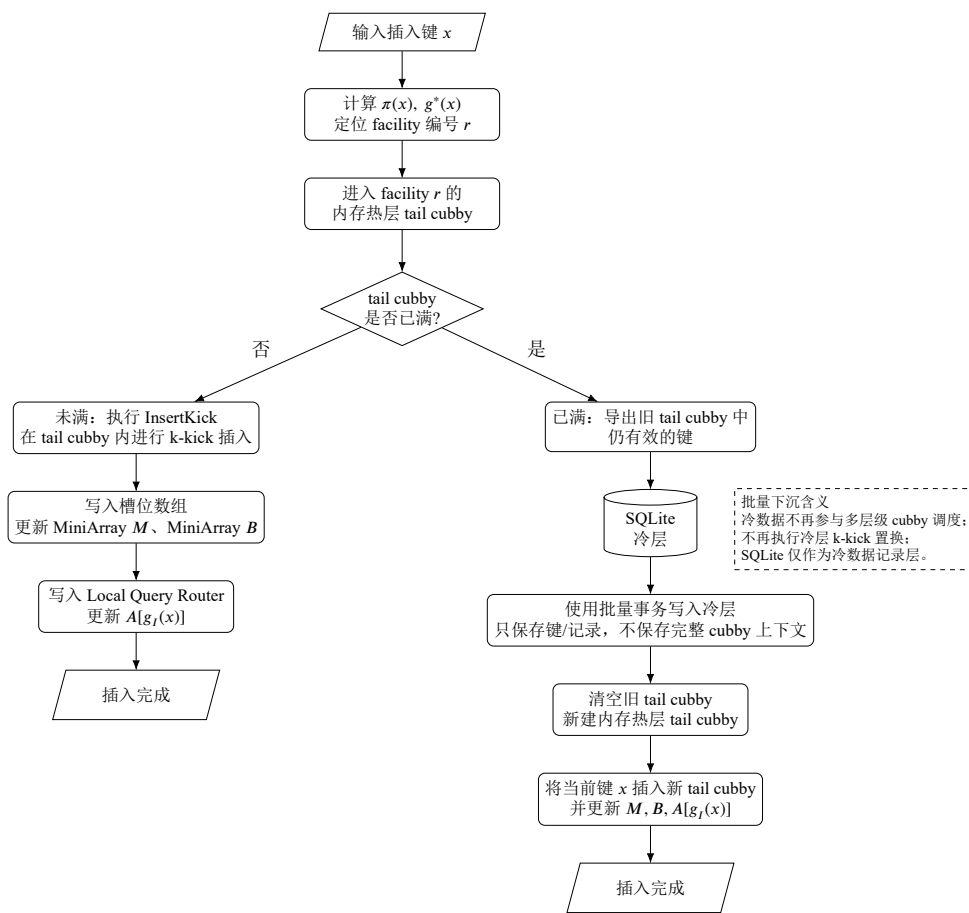


图 3-2 冷热分层结构的插入流程

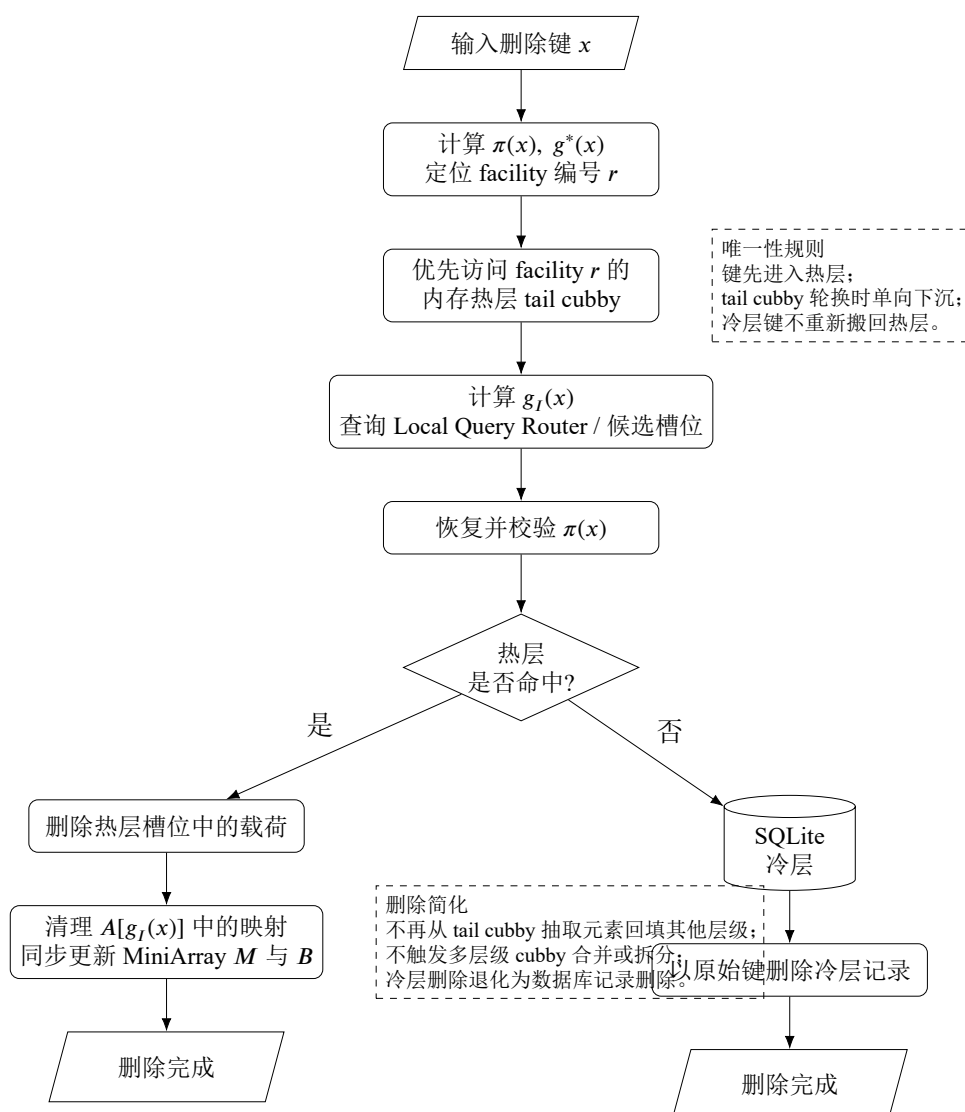


图 3-3 冷热分层结构的删除流程

第四章 系统性能实验

4.1 实验目的与环境

本章基于前两章给出的结构与参数设定，对多层级小桶哈希原型进行性能测试和功能验证。实验记录操作延迟、k-kick 踢出次数、多层级小桶向下拆分与向上合并次数，用于检查第三章各模块在实际运行中的表现^[1,5]。本章只讨论基础多层级结构本身，冷热分层改造及其对照结果放在下一章说明。

实验系统基于 C++23 标准开发，在 AMD Ryzen 7 6800H 处理器、Windows 10 环境下运行。元素规模 $n \in \{10^3, 10^4, 10^5\}$ ，facility 规模 K 按分组取多档（基准 $K = \log^3 n$ ），k-kick 最大深度 $k \in \{3, 4, 5\}$ （基准 $k = 4$ ），NODE_MAX_BITS 常数 $c = 2$ 。键由 `std::mt19937_64` 在固定随机种子 1 下均匀生成，表内哈希参数由该种子派生；实验报告单次负载内各操作类型的平均延迟。

4.2 实验方案

实验分为四组，如表 4-1 所示。规模 n 组与 K 组、 k 组采用键均匀分布于各 facility 的混合增删负载；cubby 动态重构组按纯插入、纯删除及不同插入/删除比例构造负载，分别触发向上合并与向下拆分。

表 4-1 实验分组与主要验证内容

分组	负载方式	主要验证内容
规模 n	均匀分布、混合增删	操作延迟、踢出、重构
facility K	均匀分布、混合增删	操作延迟、踢出
k-kick	均匀分布、高负载	操作延迟、踢出
cubby	插入、删除、混合比例	向下拆分、向上合并、重构耗时

实验记录的指标包括：(1) 操作延迟，即插入、命中查询和删除的平均耗时，用于观察基础操作的运行成本；(2) k-kick 踢出情况，包括一次负载内的累计踢出次数，以及单次插入、删除过程中元素搬移次数是否超过设定深度；(3) 多层

级小桶重构情况，包括向下拆分和向上合并的触发次数，用于检查层级调度是否按实现规则执行。

4.3 规模 n 实验

固定 $K \in \{64, 128, 256\}$ 、 $k = 3$ 、 $c = 2$ ，令 n 从 10^3 增至 10^5 ，各规模在三档 facility 下分别测量。表 4-2 给出全部结果。

表 4-2 规模 n 实验结果 ($K \in \{64, 128, 256\}$, $k = 3$, 键均匀分布)

n	K	插入 (μs)	查询 (μs)	删除 (μs)	总踢出	向下拆分	向上合并
10^3	64	49.4	2.1	69.7	37	16	191
	128	50.5	2.3	69.0	35	23	213
	256	52.9	2.3	69.8	13	8	215
10^4	64	47.6	3.2	82.5	822	434	2149
	128	49.1	2.8	69.4	538	295	2361
	256	49.6	2.7	70.0	325	186	2450
10^5	64	76.2	8.6	498.0	7535	10521	45018
	128	84.2	5.9	526.9	4725	10794	52417
	256	82.3	4.5	564.2	2751	11497	56349

表中向下拆分和向上合并分别对应多层级小桶数量过少或过多时触发的重构次数。

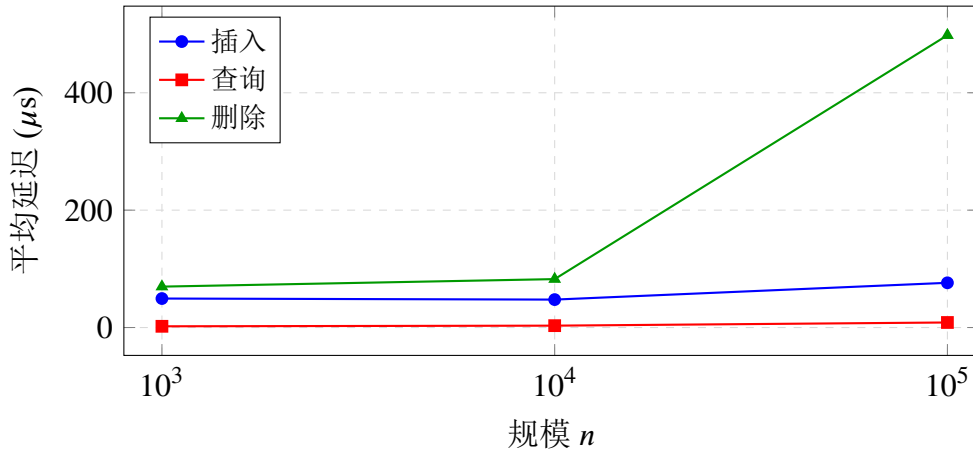


图 4-1 规模 n 实验中三种操作平均延迟 ($K = 64$, $k = 3$, 键均匀分布; 另两档 K 见表 4-2)

随 n 增大，三档 K 下命中查询延迟均缓慢上升： $n = 10^3$ 时为 2.1–2.3 μs ， $n = 10^5$ 时为 4.5–8.6 μs ，与 Local Query Router 的常数步定位一致^[5,36]；同一规模下 K 越大、facility 数越少，查询略低。插入延迟在 $n \leq 10^4$ 时三档 K 差异不大 (47.6–52.9 μs)， $n = 10^5$ 升至 76.2–84.2 μs 。删除延迟在 $n \leq 10^4$ 时稳定在

69–83 μs ; $n = 10^5$ 时升至 498–564 μs , $K = 64$ 档两类重构合计约 5.6 万次, 删除路径上的 tail cubby 回填、层级拆分及 MiniArray 同步均叠加在热路径上 (图 4-1 以 $K = 64$ 为例)。累计踢出随 n 显著增多, 且同一 n 下 K 越小踢出越多 (如 $n = 10^5$ 时由 7535 降至 2751); 向下拆分与向上合并在大规模下均为维护主要来源^[25-26]。

4.4 facility 规模 K 实验

固定 $k = 4$, 在 $n \in \{10^3, 10^4, 10^5\}$ 三档下各取多组 K 比较三操作延迟与踢出。表 4-3–4-5 汇总实测结果 (键均匀分布)。

表 4-3 facility 规模 K 实验 ($n = 10^3$, $k = 4$)

K	插入 (μs)	查询 (μs)	删除 (μs)	总踢出
16	22.6	2.1	52.8	112
64	49.9	2.0	67.7	60
256	51.9	1.9	66.0	58

表 4-4 facility 规模 K 实验 ($n = 10^4$, $k = 4$)

K	插入 (μs)	查询 (μs)	删除 (μs)	总踢出
64	56.9	2.8	75.9	813
256	54.1	2.7	70.9	916
1024	52.9	2.6	72.0	963

表 4-5 facility 规模 K 实验 ($n = 10^5$, $k = 4$)

K	插入 (μs)	查询 (μs)	删除 (μs)	总踢出
256	98.9	4.2	504.9	6889
1024	185.2	3.4	518.3	6883
4096	64.9	3.1	127.1	8756

$n = 10^3$ 时 $K = 16$ 插入最快 (22.6 μs), 但踢出最多 (112 次); $K \geq 64$ 时踢出降至约 60 次, 增删延迟略升, 体现小 n 下 facility 过细反而增加路由与维护开销。 $n = 10^4$ 三档 K 下查询均在 2.6–2.8 μs , 踢出随 K 增大变化不大 (813–963), 说明中等规模下 K 对均匀负载的边际影响有限。 $n = 10^5$ 时 $K = 4096$ 删除延迟 (127.1 μs) 明显低于 $K = 256/1024$ (504–518 μs), 与大规模重构主要落在较小 K 档、大 facility 缓解局部冲突一致; $K = 1024$ 插入异常偏高 (185.2 μs) 可能与 facility 数与 cubby 布局的组合有关, 部署时宜结合目标 n 与负载率选取 K ^[1,5]。

4.5 k-kick 深度 k 实验

对 $k \in \{3, 4, 5\}$ 在各规模下执行相同高负载混合增删负载: $n = 10^3$ 取 $K = 64$, $n = 10^4$ 取 $K = 256$, $n = 10^5$ 取 $K = 4096$ 。表 4-6–4-8 给出完整结果; 图 4-2 汇总三档规模下每键位数随 k 的变化。

表 4-6 k-kick 深度 k 实验 ($n = 10^3$, $K = 64$, 高负载)

k	插入 (μs)	查询 (μs)	删除 (μs)	总踢出
3	71.2	2.1	85.8	61
4	73.0	2.1	88.7	104
5	77.5	2.3	97.7	194

表 4-7 k-kick 深度 k 实验 ($n = 10^4$, $K = 256$, 高负载)

k	插入 (μs)	查询 (μs)	删除 (μs)	总踢出
3	54.5	2.7	72.6	265
4	55.3	2.6	72.6	900
5	56.4	2.5	70.2	1587

表 4-8 k-kick 深度 k 实验 ($n = 10^5$, $K = 4096$, 高负载)

k	插入 (μs)	查询 (μs)	删除 (μs)	总踢出
3	60.3	3.1	2229.4	2772
4	62.6	3.2	2003.9	8707
5	69.1	3.3	1734.3	14986

各档单次插入和删除过程中的最大搬移次数均不超过 k 。

三档规模下累计踢出均随 k 增大而上升: $n = 10^3$ 由 61 增至 194, $n = 10^4$ 由 265 增至 1587, $n = 10^5$ 由 2772 增至 14986^[5,9]。查询延迟在各规模均不超过 $3.3 \mu s$, 说明踢出深度主要影响增删路径。 $n = 10^5$ 删除延迟达毫秒级, 且 $k = 5$ 时 ($1734.3 \mu s$) 低于 $k = 3/4$, 这一结果与大规模重构叠加、较深踢出链在该负载下分摊部分搬移有关; 中小规模下增删延迟随 k 变化相对平缓^[1,5]。图 4-2 中每键位数随 k 增大而下降, 可作为 k 参数影响空间开销的一组实测证据。

4.6 多层次 cubby 动态重构实验

固定 $k = 4$, 在 $n \in \{10^3, 10^4, 10^5\}$ 下分别取 $K = 64, 256, 4096$, 按表 4-9–4-11 四种负载执行操作序列, 统计重构次数与单次重构平均耗时。

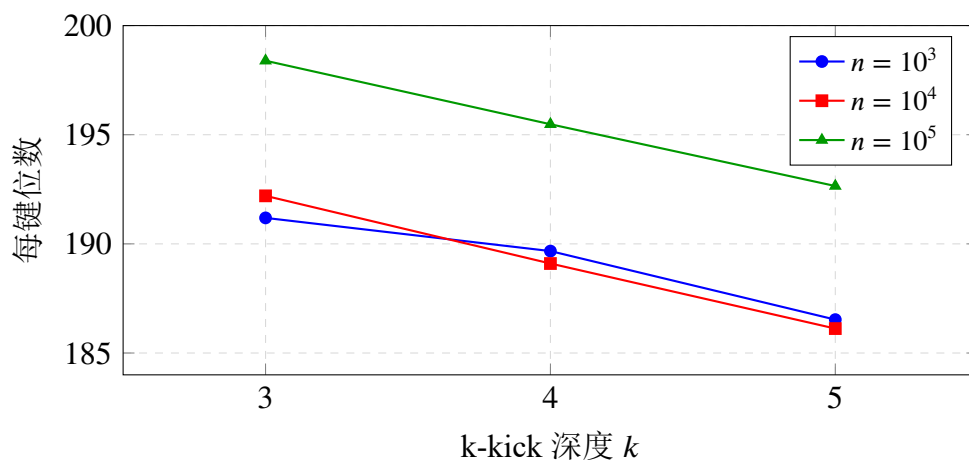


图 4-2 k-kick 深度 k 实验中每键位数

表 4-9 多层级 cubby 动态重构 ($n = 10^3$, $K = 64$)

负载	向下拆分	向上合并	平均重构耗时 (ms)
纯插入	1	31	3.72
纯删除	28	0	6.24
80/20 混合	0	24	3.71
50/50 混合	6	25	3.82

表 4-10 多层级 cubby 动态重构 ($n = 10^4$, $K = 256$)

负载	向下拆分	向上合并	平均重构耗时 (ms)
纯插入	0	384	3.47
纯删除	384	2	6.10
80/20 混合	8	355	3.32
50/50 混合	68	377	3.35

表 4-11 多层级 cubby 动态重构 ($n = 10^5$, $K = 4096$)

负载	向下拆分	向上合并	平均重构耗时 (ms)
纯插入	0	3591	5.64
纯删除	4211	621	9.76
80/20 混合	177	3588	5.52
50/50 混合	375	3504	5.13

三档规模下规律一致：纯插入以向上合并为主（ $n = 10^3$ 为 31 次， $n = 10^5$ 为 3591 次），纯删除以向下拆分为主（28–4211 次），对应小桶随元素增减而向上合并或向下拆分^[25-26]。80/20 混合负载在各规模均以向上合并为主；50/50 混合时两方向均显著（如 $n = 10^4$ 为 68/377， $n = 10^5$ 为 375/3504），删除比例升高使向下拆分占比上升。平均重构耗时由 3.3–6.2 ms（ $n \leq 10^4$ ）增至 5.1–9.8 ms（ $n = 10^5$ ），仍低于大规模混合负载中单次删除的均摊延迟。

4.7 本章小结

本章在 n 、facility 规模 K 、k-kick 深度 k 及 cubby 动态重构四组设定下，于 $n \in \{10^3, 10^4, 10^5\}$ 完成测量。表 4-2 显示，随 n 增大查询延迟由 2.1–2.3 μs 升至 4.5–8.6 μs ； $n = 10^5$ 时删除延迟为 498–564 μs ，主要受尾部回填和层级状态维护影响。

K 、 k 与 cubby 动态重构实验在表 4-3–4-11 中给出三档规模对照。各档单次搬移次数均不超过 k ，纯插入和纯删除负载分别主要触发向上合并与向下拆分。整体来看，基础多层级结构中的局部路由、有限踢出和多层级重构均按实现规则运行；删除路径的额外维护成本也为下一章讨论冷热分层改造提供了对照基础。

第五章 冷热分层存储对照实验

5.1 实验目的与方案设置

第四章验证的是基础多层级结构中小桶组织、k-kick、局部路由和重构模块能否正常工作。第三章已经说明冷热分层键存储的实现方式：近期键先进入内存 tail cubby，tail cubby 轮换时再批量下沉到 SQLite 冷层。本章在该实现基础上设置三组对照实验，观察内存 tail cubby 与 SQLite 冷数据层结合后带来的取舍。

冷热分层方案保留基础哈希结构中的分区、tail cubby 和局部路由机制，主要调整热键与冷键的数据去向。近期写入和近期访问的数据尽量留在内存中；日志结构合并树（LSM 树）采用先写内存、再批量下沉的路径^[3,28]，可以减少大量小写入直接落盘带来的成本；哈希分区则用于保留按键定位和按分区管理能力。结合第四章中的现象，tail cubby 本来就承担近期插入入口，删除近期数据时也更容易触及 tail cubby 及其回填路径。本章不再重复实现细节，而是比较这种实现与多层级内存结构、纯数据库结构之间的延迟和空间差异。

实验设置了三个对照方案：(1) 多层级内存结构，数据均保存在内存小桶中，冷数据仍由多层级小桶管理；(2) 纯数据库结构，所有插入、查询和删除都直接访问 SQLite；(3) 冷热分层结构，近期写入的数据先进入内存 tail cubby，tail cubby 写满后再批量写入 SQLite。它们使用相同的键集合、相同的分区公式和相同的统计口径，避免把输入差异误认为结构差异。

5.2 负载与统计口径

新增实验选取 5×10^3 、 10^4 、 5×10^4 、 10^5 、 5×10^5 五档规模。每一档先插入全部键，再进行同规模次数的查询，随后删除最新插入的 20% 键。查询目标按 Zipf 分布采样，排名越靠前表示越新的键，因此该负载用于模拟“写入后较快被访问”的场景^[37]。删除阶段也选择最新的 20% 键，用于观察近期数据删除时是否仍需要触发多层级维护。

统计指标包括插入平均延迟、查询平均延迟、删除平均延迟、平均每键空间占用和正确率。平均延迟只统计对应操作本身，不把收尾阶段的数据库刷盘和检查过程计入单次操作。平均每键空间占用按删除后的剩余键数计算，内存结构统计逻辑元数据位数，数据库结构统计 SQLite 主数据库文件大小。

5.3 多层级内存结构对照

表 5-1 对比了多层级内存结构与冷热分层结构。多层级内存结构的查询延迟在五档规模下均低于 $1\ \mu\text{s}$ ，说明第四章中的局部路由在新增负载下仍然保持较短查询路径。冷热分层结构的查询延迟为 $0.399\text{--}0.550\ \mu\text{s}$ ，除小规模 5×10^3 外，与多层级内存结构处于同一数量级。

表 5-1 多层级内存结构与冷热分层结构延迟对照

规模	方案	插入 (μs)	查询 (μs)	删除 (μs)
5×10^3	多层级内存结构	42.189	0.335	140.200
5×10^3	冷热分层结构	10.231	0.476	8.605
10^4	多层级内存结构	41.804	0.345	123.759
10^4	冷热分层结构	9.686	0.399	11.341
5×10^4	多层级内存结构	57.381	0.510	181.186
5×10^4	冷热分层结构	17.198	0.464	11.148
10^5	多层级内存结构	61.702	0.516	167.669
10^5	冷热分层结构	17.432	0.486	12.638
5×10^5	多层级内存结构	83.232	0.708	129.434
5×10^5	冷热分层结构	19.271	0.550	23.660

插入和删除结果更能体现两类结构的差异。多层级内存结构需要维护 tail cubby 回填以及层级合并、拆分，删除延迟在五档规模下为 $123.759\text{--}181.186\ \mu\text{s}$ ， 5×10^3 时为 $140.200\ \mu\text{s}$ 。冷热分层结构删除近期键时多发生在内存 tail cubby 或尚未完全下沉的数据区域，删除延迟为 $8.605\text{--}23.660\ \mu\text{s}$ 。这个结果说明，在本章采用的近期删除负载下，冷热分层改造减少了对多层级冷数据结构的依赖。多层级内存结构空间更紧凑；冷热分层结构将部分层级维护成本转移到 SQLite 冷层，因此不等价替代原结构。

5.4 纯数据库结构对照

表 5-2 对比了纯数据库结构与冷热分层结构。纯数据库结构的查询都需要经过 SQLite^[38]，五档规模下查询延迟为 $1.190\text{--}1.433\ \mu\text{s}$ ；冷热分层结构在相同查询

序列下为 0.399–0.550 μs 。该差异来自负载中的近期访问特征：一部分新近写入的键仍在内存 tail cubby 中，查询不必进入数据库。

表 5-2 纯数据库结构与冷热分层结构延迟对照

规模	方案	插入 (μs)	查询 (μs)	删除 (μs)
5×10^3	纯数据库结构	12.064	1.190	13.575
5×10^3	冷热分层结构	10.231	0.476	8.605
10^4	纯数据库结构	12.639	1.257	13.735
10^4	冷热分层结构	9.686	0.399	11.341
5×10^4	纯数据库结构	13.659	1.279	12.301
5×10^4	冷热分层结构	17.198	0.464	11.148
10^5	纯数据库结构	13.684	1.284	13.464
10^5	冷热分层结构	17.432	0.486	12.638
5×10^5	纯数据库结构	20.567	1.433	25.888
5×10^5	冷热分层结构	19.271	0.550	23.660

插入延迟并不是冷热分层结构在所有规模下都更低。 5×10^4 与 10^5 两档中，冷热分层结构插入分别为 17.198 μs 和 17.432 μs ，高于纯数据库结构的 13.659 μs 和 13.684 μs 。这与内存 tail cubby 中的 k-kick 插入、轮换以及后续批量写入准备有关。到 5×10^5 时，冷热分层结构插入为 19.271 μs ，略低于纯数据库结构的 20.567 μs 。删除方面，冷热分层结构五档均低于纯数据库结构，但差距不如查询明显。这组数据更接近一个边界结论：冷热分层结构主要缩短近期查询和近期删除路径，并不保证每次写入都比纯数据库结构更便宜。

5.5 空间占用与正确性对照

表 5-3 汇总三种方案的平均每键空间占用和正确率。五档规模下正确率均为 100%，说明插入、查询和删除的基本语义在新增对照实验中保持一致。空间方面，多层级内存结构为 64.427–68.305 位/键，明显低于使用 SQLite 的方案。这是因为多层级内存结构只统计逻辑元数据位数，而 SQLite 主数据库文件包含页面、表结构和索引等固定开销。

在使用 SQLite 的两类方案之间，冷热分层结构的每键位数略低于纯数据库结构，例如 10^5 时为 153.600 位/键，而纯数据库结构为 160.973 位/键。这一差异与部分近期数据仍保留在内存 tail cubby、数据库文件中实际写入量和页面分配情况有关。该结果不能简单解释为冷热分层结构空间上总是更优，因为内存 tail cubby 的逻辑元数据和数据库页面开销属于不同来源；但在本实验口径下，冷热

表 5-3 三种方案空间占用与正确率对照

规模	方案	每键位数	正确率 (%)
5×10^3	多层级内存结构	65.333	100.000
5×10^3	纯数据库结构	204.800	100.000
5×10^3	冷热分层结构	188.416	100.000
10^4	多层级内存结构	64.427	100.000
10^4	纯数据库结构	200.704	100.000
10^4	冷热分层结构	184.320	100.000
5×10^4	多层级内存结构	67.141	100.000
5×10^4	纯数据库结构	160.563	100.000
5×10^4	冷热分层结构	154.829	100.000
10^5	多层级内存结构	67.904	100.000
10^5	纯数据库结构	160.973	100.000
10^5	冷热分层结构	153.600	100.000
5×10^5	多层级内存结构	68.305	100.000
5×10^5	纯数据库结构	155.075	100.000
5×10^5	冷热分层结构	151.880	100.000

分层结构没有因为增加内存热层而超过纯数据库结构的总空间统计。

5.6 本章小结

本章用相同输入和统计口径比较了多层级内存结构、纯数据库结构和冷热分层结构。在写后读和近期删除负载下，冷热分层的主要收益来自内存热层：近期查询较少直接访问 SQLite，近期删除也较少触发多层级回填。写入并不总是占优， 5×10^4 和 10^5 两档下的插入延迟高于纯数据库结构。若查询目标大多落在冷层，或者 SQLite 已经有较高页面缓存命中率，热层判断和状态维护反而可能增加成本。因此，本章结论只对应访问局部性较强的负载，不能解释为冷热分层无条件更快。

第六章 总结与展望

6.1 工作总结

本文以 Bender 等人提出的最优时间/空间权衡框架^[5]为理论基础，完成了分层哈希表的结构与算法设计，并据此实现简化原型、开展性能实验。实现中的主要难点在于将 MiniArray、k-kick 与 Local Query Router 联调为统一系统，并在 facility-cubby-Slot 分层组织下保持辅助结构同步不变量。在此基础上，本文进一步加入冷热分层键存储方案，用内存 tail cubby 承接近期数据，用 SQLite 管理下沉后的冷数据^[3,38]。

设计部分采用 facility-cubby-Slot 分层组织。更新路径主要依赖 MiniArray 维护变长元数据，并通过 k-kick 控制插入过程中的有限搬移。查询路径由 Local Query Router 定位候选槽位，再结合商压缩信息恢复和校验键。每次发生搬移时，槽位数组、MiniArray B 、 M 与路由器都需要同步更新。

实验测量分为两部分。第四章表 4-2-4-11 及图 4-1 检查基础多层级结构的功能与维护成本；踢出深度变化时，单次搬移次数仍不超过对应的 k ，与前文交换上界一致。第五章表 5-1-5-3 对比多层级内存结构、纯数据库结构和冷热分层结构。冷热分层在近期查询和近期删除负载下延迟较低，代价是空间占用高于纯内存紧凑结构。

本文实现了 k-kick 哈希表的简化原型，记录了与严格理论模型的差异，并给出测量数据与实验参数。冷热分层部分没有完全保留原始多层级小桶的紧凑空间性质，而是把冷数据层级管理转化为数据库记录管理；这一取舍使近期数据访问路径更直接，也让持久化能力更容易接入。

6.2 不足与局限

本文实验还有四个限制：(1) 原型尚未与 IcebergHT^[25]、PaCHash^[24] 等同类系统做同机对比，因此只能说明自身参数变化下的行为，难以直接评价相对性

能；(2) 插入和删除热路径仍偏重， $n = 10^5$ 时插入、删除延迟分别为 76–84 μs 和 498–564 μs ，高于 4.5–8.6 μs 的查询延迟，主要开销来自 MiniArray 维护、k-kick 踢出和层级重构；(3) 第四章采用的合成负载较理想，规模覆盖也有限；(4) 第五章更关注写后读和近期删除，对随机冷查询、数据库页面缓冲充分以及更复杂持久化策略的覆盖还不够。

6.3 展望

后续工作优先补充偏斜负载下的尾延迟测量，并与 PaCHash 或 IcebergHT 做同机对比^[37]。完成这两项后，再补充进程级内存、数据库运行时文件和吞吐指标，同时针对插入、删除热路径做优化，使空间–时间权衡的评估口径更完整^[12,15,24]。

参考文献

- [1] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. Modern Hashing Made Simple[C/OL]// Proceedings of the 2024 SIAM Symposium on Simplicity in Algorithms (SOSA 2024). SIAM, 2024: 363-373. DOI: 10.1137/1.9781611977936.33.
- [2] CHANG F, DEAN J, GHEMAWAT S, et al. Bigtable: A Distributed Storage System for Structured Data[J/OL]. ACM Transactions on Computer Systems, 2008, 26(2): 4:1-4:26. DOI: 10.1145/1365815.1365816.
- [3] DONG S, KRYCZKA A, JIN Y, et al. RocksDB: Evolution of Development Priorities in a Key-value Store Serving Large-scale Applications[J/OL]. ACM Transactions on Storage, 2021, 17(4): 26:1-26:32. DOI: 10.1145/3483840.
- [4] 王楠, 吴云. 面向持久化键值数据库的自适应热点感知哈希索引[J/OL]. 计算机应用研究, 2024, 41(1): 226-230. DOI: 10.19734/j.issn.1001-3695.2023.04.0188.
- [5] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. On the Optimal Time/Space Tradeoff for Hash Tables[C/OL]// Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing. Rome, Italy: ACM, 2022: 1284-1297. DOI: 10.1145/3519935.3519969.
- [6] PAGH R, PAGH A, RUZIC M. Linear Probing with Constant Independence [J/OL]. SIAM Journal on Computing, 2009, 39(3): 1107-1120. DOI: 10.1137/070702278.
- [7] THORUP M, ZHANG Y. Tabulation-Based 5-Independent Hashing with Applications to Linear Probing and Second Moment Estimation[J/OL]. SIAM Journal on Computing, 2012, 41(2): 293-331. DOI: 10.1137/100800774.

- [8] CONWAY A, FARACH-COLTON M, KUSZMAUL W. A Unified Approach to Dynamic Optimality for Linear Probing and Robin Hood Hashing[C/OL]// LIPIcs: Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018): vol. 112. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018: 29:1-29:14. DOI: 10.4230/LIPIcs.ESA.2018.29.
- [9] PAGH R, RODLER F F. Cuckoo Hashing[J/OL]. Journal of Algorithms, 2004, 51(2): 122-144. DOI: 10.1016/j.jalgor.2004.03.002.
- [10] LEHMANN H P, SANDERS P, WALZER S. SicHash – Small Irregular Cuckoo Tables for Perfect Hashing[C/OL]// Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2023). SIAM, 2023: 176-189. DOI: 10.1137/1.9781611977561.ch15.
- [11] DIETZFELBINGER M, KARLIN A, MEHLHORN K, et al. Dynamic Perfect Hashing: Upper and Lower Bounds[J/OL]. SIAM Journal on Computing, 1994, 23(4): 738-761. DOI: 10.1137/S0097539791194094.
- [12] LEHMANN H P, MÜLLER T, PAGH R, et al. Modern Minimal Perfect Hashing: A Survey[J/OL]. ACM Computing Surveys, 2026, 58(10): 251:1-251:36. DOI: 10.1145/3797036.
- [13] LEHMANN H P, SANDERS P, WALZER S. ShockHash: Towards Optimal-Space Minimal Perfect Hashing Beyond Brute-Force[C/OL]// Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2024). SIAM, 2024: 194-206. DOI: 10.1137/1.9781611977929.15.
- [14] HERMANN S, LEHMANN H P, PIBIRI G E, et al. PHOBIC: Perfect Hashing with Optimized Bucket Sizes and Interleaved Coding[C/OL]// LIPIcs: Proceedings of the 32nd Annual European Symposium on Algorithms (ESA 2024): vol. 308. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024: 69:1-69:17. DOI: 10.4230/LIPIcs.ESA.2024.69.
- [15] PIBIRI G E, TRANI R. PTHash: Revisiting FCH Minimal Perfect Hashing [C/OL]// Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval. ACM, 2021: 1339-1348.

- DOI: 10.1145/3404835.3462849.
- [16] ESPOSITO E, GRAF T M, VIGNA S. RecSplit: Minimal Perfect Hashing via Recursive Splitting[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2020). SIAM, 2020: 175-185. DOI: 10.1137/1.9781611976007.14.
 - [17] LEHMANN H P, SANDERS P, WALZER S, et al. Combined Search and Encoding for Seeds, with an Application to Minimal Perfect Hashing[C/OL]//LIPIcs: Proceedings of the 33rd Annual European Symposium on Algorithms (ESA 2025): vol. 351. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025: 109:1-109:18. DOI: 10.4230/LIPIcs.ESA.2025.109.
 - [18] BLOOM B H. Space/Time Trade-offs in Hash Coding with Allowable Errors [J/OL]. Communications of the ACM, 1970, 13(7): 422-426. DOI: 10.1145/362686.362692.
 - [19] BRODER A Z, MITZENMACHER M. Network Applications of Bloom Filters: A Survey[J/OL]. Internet Mathematics, 2004, 1(4): 485-509. DOI: 10.1080/15427951.2004.10129096.
 - [20] KUSZMAUL W, WALZER S. Space Lower Bounds for Dynamic Filters and Value-Dynamic Retrieval[C/OL]//Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC 2024). ACM, 2024: 1153-1164. DOI: 10.1145/3618260.3649649.
 - [21] DIETZFELBINGER M, PAGH R. Succinct Data Structures for Retrieval and Approximate Membership[C/OL]//Lecture Notes in Computer Science: Proceedings of the 35th International Colloquium on Automata, Languages, and Programming (ICALP 2008): vol. 5125. Springer, 2008: 385-396. DOI: 10.1007/11523468_30.
 - [22] AAMAND A, KNUDSEN J B T, THORUP M. Load Balancing with Dynamic Set of Balls and Bins[C/OL]//Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC 2021). ACM, 2021: 1262-1275. DOI: 10.1145/3406325.3451107.

- [23] BANSAL N, KUSZMAUL W. Balanced Allocations: The Heavily Loaded Case with Deletions[C/OL]//Proceedings of the 63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS 2022). IEEE, 2022: 801-812. DOI: 10.1109/FOCS54457.2022.00081.
- [24] KURPICZ F, LEHMANN H P, SANDERS P. PaCHash: Packed and Compressed Hash Tables[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2023). SIAM, 2023: 162-175. DOI: 10.1137/1.9781611977561.ch14.
- [25] PANDEY P, BENDER M A, CONWAY A, et al. IcebergHT: High Performance Hash Tables Through Stability and Low Associativity[C/OL]//Proceedings of the ACM on Management of Data: Proceedings of the ACM SIGMOD International Conference on Management of Data: vol. 1: 1. Seattle, WA, USA: ACM, 2023: 47:1-47:26. DOI: 10.1145/3588727.
- [26] BENDER M A, CONWAY A, FARACH-COLTON M, et al. Iceberg Hashing: Optimizing Many Hash-Table Criteria at Once[J/OL]. Journal of the ACM, 2023, 70(6): 40:1-40:51. DOI: 10.1145/3625817.
- [27] 熊轶翔, 蒋筱斌, 张珩, 等. 支持分页显存的高性能哈希表索引系统[J/OL]. 计算机系统应用, 2022, 31(9): 82-90. DOI: 10.15888/j.cnki.csa.008664.
- [28] O'NEIL P, CHENG E, GAWLICK D, et al. The Log-Structured Merge-Tree (LSM-Tree)[J/OL]. Acta Informatica, 1996, 33(4): 351-385. DOI: 10.1007/s002360050048.
- [29] MITZENMACHER M. The Power of Two Choices in Randomized Load Balancing[J/OL]. IEEE Transactions on Parallel and Distributed Systems, 2001, 12(10): 1094-1104. DOI: 10.1109/71.963420.
- [30] LUBY M, RACKOFF C. How to Construct Pseudorandom Permutations from Pseudorandom Functions[J/OL]. SIAM Journal on Computing, 1988, 17(2): 373-386. DOI: 10.1137/0217022.
- [31] CARTER L, WEGMAN M N. Universal Classes of Hash Functions[J/OL].

- Journal of Computer and System Sciences, 1979, 18(2): 143-154. DOI: 10.1016/0022-0000(79)90044-8.
- [32] BAYER R, MCCREIGHT E M. Organization and Maintenance of Large Ordered Indexes[J/OL]. Acta Informatica, 1972, 1(1): 173-189. DOI: 10.1007/BF00288683.
- [33] PAGH R. Low Independence Hash Functions Require Logarithmic Space [J/OL]. Random Structures & Algorithms, 2004, 25(1): 22-45. DOI: 10.1002/rsa.20091.
- [34] JACOBSON G. Space-efficient Static Trees and Graphs[C/OL]//Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS 1989). IEEE, 1989: 549-554. DOI: 10.1109/SFCS.1989.63533.
- [35] 王天宇, 徐云, 王彪. 基于位图的键值存储哈希优化[J/OL]. 计算机应用研究, 2023, 40(7): 2106-2110. <http://www.c-s-a.org.cn/1003-3254/8664.html>.
- [36] MORRISON D R. PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric[J/OL]. Journal of the ACM, 1968, 15(4): 514-534. DOI: 10.1145/321479.321481.
- [37] COOPER B F, SILBERSTEIN A, TAM E, et al. Benchmarking Cloud Serving Systems with YCSB[C/OL]//Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC 2010). ACM, 2010: 143-154. DOI: 10.1145/1807128.1807152.
- [38] GAFFNEY K P, PRAMMER M, BRASFIELD L, et al. SQLite: Past, Present, and Future[J/OL]. Proceedings of the VLDB Endowment, 2022, 15(12): 3535-3547. DOI: 10.14778/3554821.3554842.

致 谢

行文至此，落笔为终。感谢我的导师、感谢身边的每一个同学、感谢遇到的每一个人。凡是过往，皆为序章，很喜欢看到的一句话，人生如棋，落子无悔。愿我们跃入人海，各自灿烂。

